Overview

A. Industry applications of natural language processing

Natural language processing (NLP) is one of the most important tasks in industry, and it makes heavy use of machine learning. NLP deals with anything related to using machines to process and understand human text/speech (i.e. “natural language”). Many companies, including giants like Google and Amazon, have teams dedicated to NLP research and application in products.

B. How NLP works

In this course, we focus more on the text aspect of NLP, since speech-related tasks will utilize many disciplines outside of machine learning (e.g. signal processing, linguistics, etc.). With written text, it is much simpler to process the data into a feasible input for a machine learning model.

Text data is normally processed with relation to a *vocabulary*. The vocabulary just is the set of unique words that appear across all the text in the *corpus* (the set of documents used to train the model). There are many different ways to process text data, but in this course you’ll learn how to convert sentences/documents into *embedding vectors*.

After processing the text data, we feed it into a particular type of neural network called a *recurrent neural network* (RNN). RNNs are great for dealing with sequential data like text, and in this course you’ll be using the long short-term memory (LSTM) variation of RNNs.

The LSTM model can be adjusted to perform various NLP tasks, ranging from text classification to text generation. For different NLP tasks, the model will have different outputs.

C. What will this course provide?

After taking this course, you’ll be able to process text data, train different

After taking this course, you'll be able to process text data, train different LSTM models on the data, and use models to perform a variety of NLP tasks.

Specifically, you will be able to:

- Process documents of text into embedding vectors
- Build a variety of different LSTm models for tasks ranging from text classification to text generation and machine translation

Introduction

An overview of natural language processing and word embeddings.

In this section, you will learn about using word embeddings to give numeric vector representations of words. Word embeddings are an extremely important part of natural language processing, since they allow us to use text with deep learning models.

A. What is NLP?

Natural language processing (NLP) encompasses any task related to machines dealing with natural language, i.e. human spoken or written language. Hence, it is one of the most important fields for machine learning and artificial intelligence. Tasks such as translating between languages, speech recognition, text analysis, and automatic text generation all fall under the scope of NLP. Without NLP we wouldn't have voice assistants like Siri and Alexa, or even search engines such as Google and Bing.

While machine learning for NLP is the focus of this course, not all NLP tasks require machine learning. For example, search engines rely largely on non-ML algorithms to find the most relevant documents or web pages for a given search query. Nevertheless, machine learning is becoming more and more widespread in NLP, and this trend will likely continue for many years to come.

B. Using text data

Natural language deals with two main categories of data: spoken or written data. While spoken language data is heavily used when building conversational agents like Siri and Alexa, written data (i.e. text data) is much more prevalent in industry NLP tasks. Raw text data is almost always unusable in NLP applications. The data is basically just a mass of strings without any real meaning for a machine to process. It is up to the engineer to first convert the raw text data into usable machine data, which can then be used as input for NLP algorithms.

{'off': 179, 'was': 20, 'like': 116, "isn't": 64, 'criminal': 163, 'falls': 122, 'late': 112, 'collide': 157,
 'passion': 184, 'bulk': 83, 'have': 193, 'many': 224, 'solid': 151, '1987': 79, 'guys': 49, 'credited': 107,
 'lost': 153, "could've": 186, 'recalled': 80, 'it': 5, 'least': 73, 'kenneth': 85, 'traditional': 190, 'betw
een': 161, "that's": 134, 'you': 140, 'often': 120, 'without': 21, 'contact': 169, 'my': 19, 'expressing': 22
1, "it's": 40, 'our': 175, 'car': 136, 'ships': 154, 'in': 6, 'making': 39, 'another': 127, 'secretary': 162,
 'to': 12, 'he': 34, 'at': 72, 'lots': 52, 'different': 155, 'wanted': 173, 'star': 65, 'though': 197, 'movie
s': 211, 'its': 124, 'feeling': 227, 'rml': 185, 'beautiful': 216, 'polluted': 58, 'for': 150, 'age': 128, 'd
eeply': 141, 'nolte': 218, 'is': 7, 'minutes': 192, 'glad': 105, 'from': 18, 'any': 167, 'directed': 28, 'dec
ide': 172, 'depths': 57, 'carradine': 68, 'once': 74, 'done': 187, 'duds': 203, 'type': 130, 'ever': 212, 'se
ries': 198, 'fun': 63, 'glenn': 31, 'pulls': 139, 'bit': 206, 'john': 67, 'great': 47, 'result': 103, 'makes'
: 26, 'simply': 182, 'rises': 56, 'very': 131, 'how': 101, 'toward': 170, 'vonneguty': 219, 'awful': 102, 'in
trigued': 149, 'and': 2, 'library': 202, 'blockbuster': 200, 'tv': 114, 'netflix': 199, 'just': 138, 'his': 9
0, 'portraying': 92, 'effective': 132, 'faith': 205, 'consummate': 183, 'interview': 76, "you'd": 60, 'footag
e': 88, 'strock': 27, 'leave': 117, 'hartford': 86, 'restored': 204, 'tony': 66, '100': 191, 'heroine': 177,
"couldn't": 171, 'book': 10, 'let': 146, 'every': 119, 'fangoria': 78, 'the': 1, 'does': 24, 'greatest': 210,
 'sexual': 158, 'had': 82, 'been': 35, 'main': 142, 'credit': 99, 'end': 33, 'night': 230, 'an': 75, 'which':
 36, 'women': 51, 'but': 9, 'characters': 32, 'thing': 133, 'palpable': 165, 'face': 125, 'job': 220, 'lake':
 59, 'would': 22, 'received': 97, 'thin': 207, 'made': 213, 'chase': 137, 'of': 4, 'films': 225, 'eisley': 25
, 'after': 46, 'reading': 44, 'this': 41, 'engage': 226, 'almost': 164, 'featuring': 89, 'so': 38, 'most': 54
, 'tough': 48, 'curious': 188, 'why': 115, 'only': 29, 'nick': 217, 'caper': 180, 'on': 37, 'into': 42, 'moth
er': 229, 'one': 209, 'swearing': 53, 'that': 8, 'hero': 176, 'reminder': 126, "i'm": 195, 'hardly': 166, 'ha
s': 108, 'alone': 147, 'entertainment': 121, 'hours': 152, 'children': 91, 'particularly': 94, 'even': 43, 's
omehow': 84, 'worth': 70, 'flat': 123, 'lee': 215, 'viewing': 71, 'film': 11, 'monster': 17, 'their': 45, 'th
ink': 61, 'cover': 145, 'feel': 110, 'be': 62, 'mid': 111, 'pull': 178, 'probably': 104, 'herbert': 81, 'back
ground': 118, 'as': 15, 'feelings': 222, 'two': 16, 'unconvincing': 55, 'local': 201, 'or': 181, 'sexy': 50,

Example of processed text data.

In the following chapters you'll be introduced to an easy and efficient way to process raw text, and then you'll use the processed text to run a machine learning algorithm.

Vocabulary

Become accustomed to the meaning of "vocabulary" for NLP tasks.

Chapter Goals:

- Learn about the text corpus and vocabulary in NLP tasks
- Create a function that tokenizes a text corpus

A. Corpus vocabulary

In the context of NLP tasks, the **text corpus** refers to the set of texts used for the task. For example, if we were building a model to analyze news articles, our text corpus would be the entire set of articles or papers we used to train and evaluate the model.

The set of unique words used in the text corpus is referred to as the **vocabulary**. When processing raw text for NLP, everything is done around the vocabulary.

```
print(text_corpus) # a list of different texts (sentences)
print(vocabulary)  # a list of the words that make up those texts
```

Example text corpus and its corresponding vocabulary.

In addition to using the words of a text corpus as the vocabulary, you could also use a **character-based** vocabulary. This would consist of each unique character in the text corpus (e.g. each letter). In this course, we'll be focusing on **word-based vocabularies**, which are much more common than their character-based counterparts.

B. Tokenization

We can use the vocabulary to find the number of times each word appears in the corpus, figure out which words are the most common or uncommon, and

the corpus, figure out which words are the most common or uncommon, and filter each text document based on the words that appear in it. However, the

most important part of the vocabulary is that it allows us to represent each piece of text by the specific words that appear in it.

Rather than being represented as one long string, a piece of text can be represented as a vector/list of its vocabulary words. This process is known as **tokenization**, where each individual vocabulary word in a piece of text is a **token**.

Below we show an example of tokenization on a text corpus.

```
print(text_corpus)      # a list of texts
print(processed_corpus) # the texts broken down into lists of vocabulary words
```

Representing pieces of text by vocabulary words. Note that the punctuation is filtered out.

In the example above, the punctuation is filtered out of the text corpus. While it is normally standard to filter out punctuation, in some cases (e.g. generating long text) it may be necessary to keep punctuation in the vocabulary. It is a good idea to understand the NLP task you are going to perform before filtering out any data/piece of text.

C. Tokenizer object

Using TensorFlow, we can convert a text corpus into tokenized sequences using the Tokenizer object. The `Tokenizer` class is part of the `tf.keras` submodule, which is TensorFlow's implementation of Keras, a high-level API for machine learning.

The `Tokenizer` object contains the functions `fit_on_texts` and `texts_to_sequences`, which are used to initialize the object with a text corpus and convert pieces of text into sequences of tokens, respectively.

```
import tensorflow as tf
tokenizer = tf.keras.preprocessing.text.Tokenizer()
text_corpus = ['bob ate apples, and pears', 'fred ate apples!']
tokenizer.fit_on_texts(text_corpus)
new_texts = ['bob ate pears', 'fred ate pears']
print(tokenizer.texts_to_sequences(new_texts))
```

```
print(tokenizer.word_index)
```

Using a Tokenizer object for text processing. Notice that it maps vocabulary words to indexes.

The `Tokenizer` automatically converts each vocabulary word to an integer ID (IDs are given to words by descending frequency). This allows the tokenized sequences to be used in NLP algorithms (which work on vectors of numbers). In the above example, the `texts_to_sequences` function converts each vocabulary word in `new_texts` to its corresponding integer ID.

D. Tokenizer parameters

The `Tokenizer` object can be initialized with a number of optional parameters. By default, the `Tokenizer` filters out any punctuation and white space. You can specify custom filtering with the `filters` parameter. The parameter takes in a string, where each character in the string is filtered out.

When a new text contains words not in the corpus vocabulary, those words are known as out-of-vocabulary (OOV) words. The `texts_to_sequences` automatically filters out all OOV words. However, if we want to specify each OOV word with a special vocabulary token (e.g. `'OOV'`), we can initialize the `Tokenizer` with the `oov_token` parameter.

```
import tensorflow as tf
tokenizer = tf.keras.preprocessing.text.Tokenizer(
  oov_token='OOV')
text_corpus = ['bob ate apples, and pears', 'fred ate apples!']
tokenizer.fit_on_texts(text_corpus)
print(tokenizer.texts_to_sequences(['bob ate bacon']))
print(tokenizer.word_index)
```

Using a Tokenizer with 'OOV' as the OOV token. The text corpus is the same one from the previous example. In this example, 'bacon' is not in the corpus vocabulary.

The `num_words` parameter lets us specify the maximum number of vocabulary words to use. For example, if we set `num_words=100` when initializing the `Tokenizer`, it will only use the 100 most frequent words in the vocabulary and filter out the remaining vocabulary words. This can be useful when the text

filter out the remaining vocabulary words. This can be useful when the text

corpus is large and you need to limit the vocabulary size to increase training speed or prevent overfitting on infrequent words.

```
import tensorflow as tf
tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=2)
text_corpus = ['bob ate apples, and pears', 'fred ate apples!']
tokenizer.fit_on_texts(text_corpus)

# the two most common words are 'ate' and 'apples'
# the tokenizer will filter out all other words
# for the sentence 'bob ate pears', only 'ate' will be kept
# since 'ate' maps to an integer ID of 1, the only value
# in the token sequence will be 1
print(tokenizer.texts_to_sequences(['bob ate pears']))
```

Using a Tokenizer with a vocabulary size of 2. Since 'bob' and 'pears' are not two of the most frequent words in the corpus, they are filtered out.

Time to Code!

The code for this section of the course involves building up an embedding model. Specifically, you will be building out the `EmbeddingModel` object. In this chapter, you'll be completing the `tokenize_text_corpus` function.

You'll notice that in the model initialization, the `Tokenizer` object is already set, with its maximum vocabulary size set to `vocab_size`. However, the `Tokenizer` object has not yet been initialized with a text corpus.

In the `tokenize_text_corpus` function, we'll first initialize the `Tokenizer` with the text corpus, `texts`.

Call `self.tokenizer.fit_on_texts` on `texts`.

After initializing the `Tokenizer` with the text corpus, we can use it to convert the text corpus into tokenized sequences.

Set `sequences` equal to `self.tokenizer.texts_to_sequences` applied to `texts`. Then return `sequences`.

```
import tensorflow as tf
```

```
# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Convert a list of text strings into word sequences
    def tokenize_text_corpus(self, texts):
        # CODE HERE
        pass
```

Embeddings

Learn the basics of word embeddings and why they're used.

Chapter Goals:

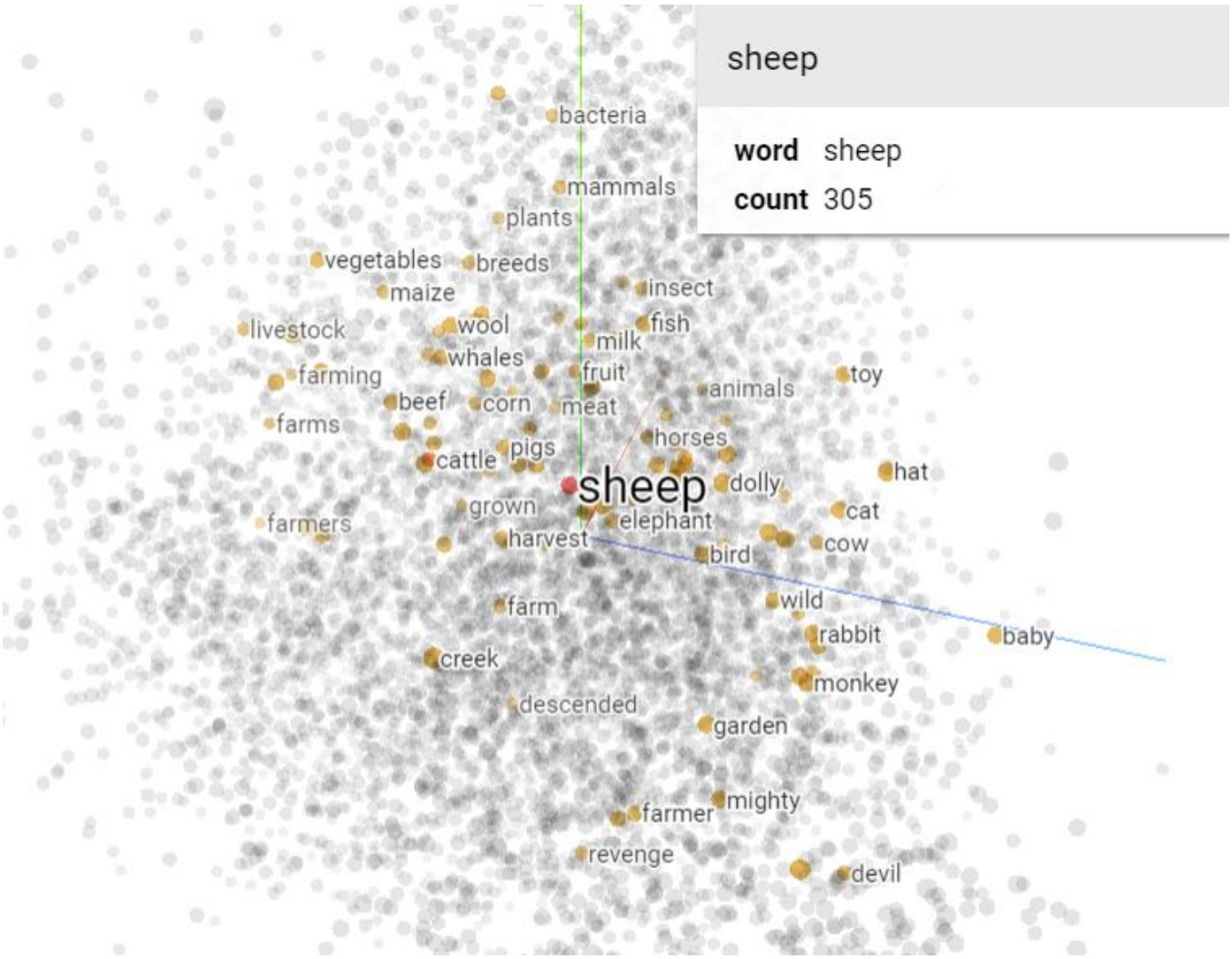
- Learn about word embeddings and why they're used
- Create a function that retrieves a target word and its context

A. Word representation

So far, the way we've represented vocabulary words is with unique integer IDs. However, these integer IDs don't give a sense of how different words may be related. For example, if a vocabulary gave the words "milk" and "cereal" the IDs 14 and 103, respectively, we would have no idea that these two words are often used within the same context.

The solution to this problem is to convert each word into an *embedding vector*. An embedding vector is a higher-dimensional vector representation of a vocabulary word. Since vectors have a sense of distance (as they are just points in a higher-dimensional space), embedding vectors give us a word representation that captures relationships between words.

Below is an example of word embedding vectors from the TensorFlow Embedding Projector. Note that the words most similar to "sheep" have shorter pointwise distances.



When creating embedding vectors for your vocabulary, something to consider is how large the vectors are (i.e. the number of dimensions). Larger vectors are able to capture more relational tendencies between words, and are therefore better if you have a large vocabulary size. However, they also use up more computational resources and are more likely to overfit on smaller vocabularies.

A general rule of thumb is to set the number of dimensions in your embedding vectors equal to the 4th root of the vocabulary size. For example, if your vocabulary consists of 10000 words, the rule says your embedding vectors should have 10 dimensions. However, similar to the batch size or number of hidden layers in a neural network, you should try different embedding sizes to figure out which is best for the task at hand.

B. Target-context

The basis for embedding vectors comes from the concept of a *target* word and its *context window*. For each word of each tokenized sequence in the text

corpus, we treat it as a *target*, and the words around it as the *context window*.

This is how we define embedding vector relations; words that often appear near each other tend to be related.

A target word ("peanut") and context window (outlined in green).

We refer to the size of the context window as the *window size*. Since we want the context to be symmetric on both sides of the target word, the window size should be an odd number. In the above example, the window size is 5 (two words to the left and right of the target word).

We'll be learning how to convert target words and context windows into data that can be used to train an embedding vector model in the next chapter.

Time to Code!

In this chapter, you'll be completing the `get_target_and_context` function. This involves completing several helper functions.

The first helper function to complete is `get_target_and_size`. This returns a tuple containing the target word and half the window size.

The target word is located at `target_index` of `sequence`.

Set `target_word` equal to the element located at `target_index` of the list `sequence`.

Since the window's context is symmetric around the target word, the context will have length equal to half the window size (rounded down) on either side of the target word.

Set `half_window_size` equal to `window_size` divided by `2`, rounded down.

Then return a tuple with `target_word` as the first element and `half_window_size` as the second element.

```
import tensorflow as tf

def get_target_and_size(sequence, target_index, window_size):

    # CODE HERE
    pass

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Convert a list of text strings into word sequences
    def get_target_and_context(self, sequence, target_index, window_size):
        target_word, half_window_size = get_target_and_size(
            sequence, target_index, window_size
        )
```

Next, you'll complete the helper function `get_window_indices`. This returns a tuple containing the left (inclusive) and right (exclusive) indices of the window.

To match Pythonic `range` indexing, we set the window's start index as inclusive and the stop index as exclusive. We also need to make sure the window does not extend out of the indexable range of `sequence`.

Set `left_incl` equal to the maximum between `0` and `target_index - half_window_size`.

Set `right_excl` equal to the minimum between the length of `sequence` and `target_index + half_window_size + 1`.

Then return a tuple with `left_incl` as the first element and `right_excl` as the second element.

```
import tensorflow as tf

def get_window_indices(sequence, target_index, half_window_size):
    # CODE HERE
    pass

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
```

```
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Convert a list of text strings into word sequences
    def get_target_and_context(self, sequence, target_index, window_size):
        target_word, half_window_size = get_target_and_size(
            sequence, target_index, window_size
        )
        left_incl, right_excl = get_window_indices(
            sequence, target_index, half_window_size)
        return target_word, left_incl, right_excl
```

Skip-gram

Understand the difference between skip-gram and CBOW embedding models.

Chapter Goals:

- Learn about the types of models used to create embeddings
- Understand the difference between the skip-gram and CBOW models

A. Embedding models

In the previous chapter, we discussed how embedding vectors are based on target words and context windows. Specifically, we use these target words and context windows to create training *pairs*. Depending on the type of embedding model we use, the pairs will either be target-context pairs or context-target pairs.

Skip-gram

The skip-gram model uses target-context training pairs. Each pair has a target word as the first element and a context word as the second element. This means that every context window will produce multiple training pairs. For example, if the context window is "paul likes **singing** in french", (where **singing** is the target word), the training pairs would be

(singing, paul), (singing, likes)
(singing, in), (singing, french)

When training an embedding model, we convert the first element in each pair (for the skip-gram model, this is the target word) into its corresponding embedding vector. This will be discussed in greater depth in later chapters.

CBOW

The continuous-bag-of-words (CBOW) model uses context-target training pairs. Each pair will have all the context words as the first element and the target word as the second element. For example, if the context window is "tom

eats **spicy** crab salad" (where **spicy** is the target word), the training pair would be

$$((\text{tom}, \text{eats}, \text{crab}, \text{salad}), \text{spicy})$$

Since the context element for the CBOW model contains multiple words, we use the average context embedding vector when training our embedding model. So in the example above, the context embedding vector would be the average between the four embedding vectors for "tom", "eats", "crab", and "salad".

B. Skip-gram vs. CBOW

When it comes to choosing between the skip-gram and CBOW embedding models, there are a few things to consider. Since the skip-gram model creates a training pair for each context word, it requires much less actual data than the CBOW model. However, this also means that the CBOW model is faster to train.

Furthermore, since the skip-gram model is creating multiple instances of training pairs for each target word, it can represent rarer words or phrases better than the CBOW model. On the other hand, the CBOW model provides more accurate embeddings for more common words.

In this course, the training data is small, so we will create a skip-gram model. However, when creating your own personal embedding model for an NLP task, it's best to take into account all the factors when making a decision.

Time to Code!

In this chapter, you'll be completing the `create_target_context_pairs` function, which creates target-context pairs for the skip-gram embedding model. Specifically, you'll be writing code in the portion of the function that says **"CODE HERE"**.

The target-context pairs we create will all be tuples containing `target_word` as the first element. The second element will be the context word ID at each context window index.

At each index, `j`, in

`range(left_incl, right_excl)`

where `j != i`, append the tuple

`(target_word, sequence[j])`

to the end of `pairs`.

```
import tensorflow as tf

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    def tokenize_text_corpus(self, texts):
        self.tokenizer.fit_on_texts(texts)
        sequences = self.tokenizer.texts_to_sequences(texts)
        return sequences

    # Convert a list of text strings into word sequences
    def get_target_and_context(self, sequence, target_index, window_size):
        target_word = sequence[target_index]
        half_window_size = window_size // 2
        left_incl = max(0, target_index - half_window_size)
        right_excl = min(len(sequence), target_index + half_window_size + 1)
        return target_word, left_incl, right_excl

    # Create (target, context) pairs for a given window size
    def create_target_context_pairs(self, texts, window_size):
        pairs = []
        sequences = self.tokenize_text_corpus(texts)
        for sequence in sequences:
            for i in range(len(sequence)):
                target_word, left_incl, right_excl = self.get_target_and_context(
                    sequence, i, window_size)
                # CODE HERE
        return pairs
```

Embedding Matrix

Create a trainable embedding matrix to calculate word embeddings.

Chapter Goals:

- Learn how the embedding matrix is initialized during training
- Create a function for the forward run of embedding training

A. Variable creation

In TensorFlow, we use the `tf.layers.dense` function to create each layer of a fully-connected neural network. This is a function provided by TensorFlow that automatically handles all the setup required for the neural network's weights. So previously all we needed to do was initialize the weight variables (via `tf.global_variables_initializer`) prior to running the computation graph.

However, for our embedding model, the variable we need to create is a 2-D matrix that contains the embedding vectors for each vocabulary word ID. Therefore, we need to manually create this variable using the `tf.get_variable` function.

The function only takes in one required argument, which is the name of the variable. Of the possible keyword arguments, a couple important ones to know are `shape` and `dtype`, which allow us to specify the shape and type, respectively, of the created variable.

Below we demonstrate example usages of `tf.get_variable`:

```
import tensorflow as tf
print(tf.get_variable('v1', shape=(1, 3)))
print(tf.get_variable('v2', shape=(2,), dtype=tf.int64))
```

In the example above, the first variable is created with just the `shape` keyword argument (so its default type is `tf.float32`), while the second is manually set to type `tf.int64` (the `_ref` suffix does not affect the type).

During training, the first call to `tf.get_variable` with a given variable name will create a new variable. Each subsequent call to the function using the same variable name will retrieve the existing variable, rather than create a new one. This allows us to continue training with the same embedding matrix.

B. Variable initializers

Another useful keyword argument to know for `tf.get_variable` is the `initializer` argument, which is used to specify how the variable will be initialized. The argument can also be used to specify the variable's shape (in which case we don't need to set the `shape` keyword argument).

The default initializer for variables created by `tf.get_variable` is a `glorot_uniform_initializer`. However, we can use our own probability distribution to randomly initialize the variable. A popular probability distribution to use is `tf.random_uniform`, corresponding to a uniform distribution.

Below we demonstrate an example usage of `tf.random_uniform` to initialize a variable:

```
import tensorflow as tf
init = tf.random_uniform((5, 10),minval=-1,maxval=2)
v = tf.get_variable('v1', initializer=init)
```

In the example above, we used `tf.random_uniform` to return a tensor with shape `(5, 10)`, containing randomly chosen values from the uniform distribution in the range [-1, 2]. We then used `init` to initialize `v` with its randomly chosen values.

C. Embedding lookup

When training the embedding model, the "forward" run consists of variable initialization/retrieval followed by *embedding lookup* for the current

initialization/retrieval followed by *embedding lookup* for the current iteration's training batch. Embedding lookup refers to retrieving the

embedding vectors for each word ID in the training batch. Since the embedding matrix's rows are each unique embedding vectors, we perform the lookup simply by retrieving the rows corresponding to the training batch's word IDs.

The function we use to retrieve the embedding vectors is `tf.nn.embedding_lookup`. It takes in two required arguments, which are the embedding matrix variable and vocabulary IDs to lookup.

Below we demonstrate an example usage of `tf.nn.embedding_lookup`:

```
import tensorflow as tf
emb_mat = tf.get_variable('v1', shape=(5, 10))
word_ids = tf.constant([0, 3])
emb_vecs = tf.nn.embedding_lookup(emb_mat, word_ids)
print(emb_vecs)
```

In the example above, we used `tf.nn.embedding_lookup` to retrieve embedding vectors from the embedding matrix, `emb_mat`, for the word IDs `0` and `3`. The output tensor from the embedding lookup will contain the embedding vector for ID `0` in the first row, and the embedding vector for ID `3` in the second row.

Time to Code!

In this chapter, you'll be completing the `forward` function, which retrieves embeddings using an embedding matrix.

First, you'll complete the helper function `get_initializer`. This function returns the initializer for the embedding matrix.

We're going to initialize our embedding matrix variable with a random uniform initializer. As a rule of thumb, we'll set the uniform distribution bounds to [-0.5/d, 0.5/d), where d is the embedding vector size (i.e. `embedding_dim`).

Set `initial_bounds` equal to `0.5` divided by `embedding_dim`.

We want our embedding matrix to have shape `(vocab_size, embedding_dim)`, since each row of the matrix corresponds to an embedding vector for a unique vocabulary word.

Set `initializer` equal to `tf.random_uniform` with the specified matrix shape as the required argument.

The `minval` and `maxval` keyword arguments should correspond to negative and positive `initial_bounds`, respectively.

```
import tensorflow as tf

def get_initializer(embedding_dim, vocab_size):
    # CODE HERE
    pass

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Forward run of the embedding model to retrieve embeddings
    def forward(self, target_ids):
        initializer = get_initializer(
            self.embedding_dim, self.vocab_size)
```

Now you'll work in the main function, `forward` (where it says **"CODE HERE"**). Using the initializer, we can create/retrieve the embedding matrix variable, then use it to get embeddings.

Set `self.embedding_matrix` equal to `tf.get_variable` with `'embedding_matrix'` as the required argument and `initializer` as the `initializer` keyword argument.

Set `embeddings` equal to `tf.nn.embedding_lookup` with `self.embedding_matrix` and `target_ids` as the two required arguments. Then return `embeddings`.

```
import tensorflow as tf
```

```
# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Forward run of the embedding model to retrieve embeddings
    def forward(self, target_ids):
        initializer = get_initializer(
            self.embedding_dim, self.vocab_size)
        # CODE HERE
```

Candidate Sampling

Understand why candidate sampling is used for embedding training.

Chapter Goals:

- Learn about candidate sampling and why it is useful for embedding training

A. Large vocabularies

To obtain good word embeddings, it is usually necessary to train an embedding model on a large amount of text data. This means that the vocabulary size will likely be very large, often reaching tens of thousands of words. However, having a large vocabulary size can significantly slow down training.

Training an embedding model is equivalent to multiclass classification, where the possible classes include every single vocabulary word. This means that we would need to calculate a softmax loss across every single vocabulary word during training, which can be incredibly time consuming for large vocabularies.

In order to mitigate the costly full softmax operation, we apply something called *candidate sampling*. With candidate sampling, we choose a much smaller fraction of the possible classes (i.e. vocabulary words) for computing the loss. This speeds up training significantly while also maintaining performance if we use the proper candidate samplers and loss function (more on this in the next chapter).

B. Computing logits

When we calculate the loss for an embedding model, we need to first compute the model's logits. We do this by setting up trainable weights and bias terms, which will be variables created by the `tf.get_variable` function.

Similar to the final layer of a multilayer perceptron, which computes the

MLP's logits, the weights and bias variables for the embedding model also

compute the logits, which are then converted into the loss based on the loss function.

Time to Code!

In this chapter, you'll be completing the `get_bias_weights` function, which gets the bias and weights for calculating the embedding loss (next chapter).

In order to calculate the loss for our candidate sampling algorithm, we need to create weight and bias variables. The weight variable will have shape `[self.vocab_size, self.embedding_dim]`, while the bias variable will have shape `[self.vocab_size]`.

We'll initialize the values for both the weight and bias variables to all 0's.

Set `weights_initializer` equal to `tf.zeros` applied with the weight variable's shape as the only argument.

Set `bias_initializer` equal to `tf.zeros` applied with the bias variable's shape as the only argument.

Next we'll create the weight and bias variables using these initializers and the `tf.get_variable` function.

Set `weights` equal to `tf.get_variable` with `'weights'` as the required argument `weights_initializer` as the `initializer` keyword argument.

Set `bias` equal to `tf.get_variable` with `'bias'` as the required argument `bias_initializer` as the `initializer` keyword argument.

Return a tuple with `weights` as the first element and `bias` as the second element.

```
import tensorflow as tf

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)
```

```
    # Get bias and weights for calculating loss
    def get_bias_weights(self):

        # CODE HERE
        pass
```

Embedding Loss

Learn about different loss functions for embedding training.

Chapter Goals:

- Learn about the different types of candidate sampling algorithms and loss functions
- Calculate the embedding model's loss with the NCE loss function

A. Loss functions

As mentioned in the previous chapter, candidate sampling avoids performing a costly full softmax operation to calculate the embedding loss. Instead, there are two main loss functions we use: sampled softmax and NCE loss.

Sampled Softmax

As the name suggests, this is just a softmax loss with "sampled" classes. The classes we use to calculate the softmax include the actual context vocabulary word (the true label), as well as a randomly chosen set of words from the entire vocabulary to act as the false labels. In TensorFlow, we can compute the sampled softmax loss using the `tf.nn.sampled_softmax_loss` function.

dog	cat	pun	bowl	ice	four
0.70	0.18	0.00	0.08	0.00	0.04

Sampled softmax example for target word "growl". The actual context word (dog) and sampled words are shown with their corresponding softmax probabilities underneath.

For training purposes, the sampled softmax loss gives a good approximation of the full softmax loss, while taking a fraction of the time to calculate. However, when evaluating a trained model's loss on a validation or test set, it's best to use the full softmax cross entropy loss for the most accurate

metrics.

NCE Loss

The NCE (noise-contrastive estimation) loss takes a different approach than the sampled softmax. The idea is to convert the multiclass classification problem into a binary classification problem.

With the softmax-based loss functions, the embedding model is trained to predict the singular correct context word out of an entire set of vocabulary words. This can be a difficult task for the model to perform, even with a smaller sampled vocabulary set.

With noise-contrastive estimation, the model instead uses a sigmoid-based probabilistic approach. So for each of the sampled words, the model would want to output a low probability that the sampled word is part of the target word's context. On the other hand, the model should output a high probability for the true context word.

dog	cat	pun	bowl	ice	four
0.95	0.40	0.05	0.22	0.01	0.17

NCE probability example, using the same target, context, and sampled words as the previous example. Note that the probabilities don't sum to 1, since the sigmoid probabilities are independent of one another.

This approach is a much simpler task than calculating softmax probabilities, and is therefore normally favored when training an embedding model. In TensorFlow, we can compute the NCE loss using the `tf.nn.nce_loss` function.

Similar to the sampled softmax loss, the NCE loss provides a good loss approximation during training, but should be replaced by the full sigmoid cross entropy loss during evaluation or testing for the most accurate metrics.

Time to Code!

In this chapter, you'll be completing the `calculate_loss` function, which calculates the embedding loss.

To calculate the loss for our embedding model, we'll use the NCE loss function

To calculate the loss for our embedding model, we'll use the NCE loss function, `tf.nn.nce_loss`. The function takes in six required arguments:

- **weights:** The weights for computing the model's logits.
- **biases:** The bias terms for computing the model's logits.
- **labels:** The training batch's labels. For our skip-gram model, this will be the context IDs.
- **inputs:** The input embeddings for the NCE loss. For our skip-gram model, these are the target ID embeddings.
- **num_sampled:** The number of negative classes to sample.
- **num_classes:** The number of classes for calculating the loss. For embedding models, this is the vocabulary size.

The function's output is the NCE loss for each target-context pair in the training batch.

Set `nce_losses` equal to `tf.nn.nce_loss` applied with `weights`, `bias`, `context_ids`, `embeddings`, `num_negative_samples`, and `self.vocab_size` as the six required arguments.

We aggregate the NCE losses for the training batch into the overall loss, by taking the average across each of the NCE losses.

Set `overall_loss` equal to `tf.reduce_mean` applied to `nce_losses`. Then return `overall_loss`.

```
import tensorflow as tf

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Get bias and weights for calculating loss
    def get_bias_weights(self):
        weights_initializer = tf.zeros([self.vocab_size, self.embedding_dim])
        bias_initializer = tf.zeros([self.vocab_size])
        weights = tf.get_variable('weights',
            initializer=weights_initializer)
        bias = tf.get_variable('bias',
            initializer=bias_initializer)
        return weights, bias

    # Calculate NCE Loss based on the retrieved embedding and context
```

```
    def calculate_loss(self, embeddings, context_ids, num_negative_samples):
        weights, bias = self.get_bias_weights()
        # CODE HERE
```

Cosine Similarity

Implement normalized cosine similarity to evaluate the embedding model.

Chapter Goals:

- Learn about cosine similarity and how it's used to compare embedding vectors
- Create a function that computes cosine similarities for a given word

A. Vector comparison

In mathematics, the standard way for comparing vector similarity is through *cosine similarity*. Since word embeddings are just vectors of real numbers, we can use also cosine similarity to compare embeddings for different words.

For two vectors, $\mathbf{u}$ and $\mathbf{v}$, the equation for cosine similarity is

$$\cos \text{sim} = \frac{\mathbf{u}}{||\mathbf{u}||_2} \cdot \frac{\mathbf{v}}{||\mathbf{v}||_2}$$

where $||v||_2$ represents the L2-norm of vector v, and $\cdot$ represents the dot product operation.

We refer to the quantity $\frac{v}{||v||_2}$ as the L2-normalization of vector v.

B. Correlation

The cosine similarity measures the *correlation* between two vectors, i.e. how closely related the two vectors are. The range of values for cosine similarity is [-1, 1]. A value of 1 means the vectors are perfectly identical, a value of -1 means the vectors are complete opposites, and a value of 0 means the vectors are *orthogonal* (i.e. completely uncorrelated).

Vector pairs showcasing cosine similarities of -1 (left), 1 (center), and 0 (right).

Note that the cosine similarity values are based on a spectrum, so we can measure correlation based on the cosine similarity's proximity to 1, 0, or -1. For example, we would expect the word embeddings for "orange" and "juice" to have a cosine similarity close to 1, since they are often used in the same context in conjunction with one another. On the other hand, we would expect "good" and "bad" to have a negative cosine similarity, since they are antonyms. And in most text corpuses, "chocolate" and "fence" would have a cosine similarity near 0, since they tend to be unrelated.

For example, imagine two vectors v_1 and v_2.

$$v_1 = \begin{bmatrix} 2 \\ 0 \\ 6 \end{bmatrix}, v_2 = \begin{bmatrix} 4 \\ 2 \\ 5 \end{bmatrix}$$

The L2 norm of v_1 is $\sqrt{2^2 + 0^2 + 6^2} = 6.324$

The L2 norm of v_2 is $\sqrt{4^2 + 2^2 + 5^2} = 6.708$

$$\frac{v_1}{6.324} = \begin{bmatrix} 0.316 \\ 0 \\ 0.949 \end{bmatrix}$$

$$\frac{v_2}{6.708} = \begin{bmatrix} 0.596 \\ 0.298 \\ 0.745 \end{bmatrix}$$

$$\begin{bmatrix} 0.316 \\ 0 \\ 0.949 \end{bmatrix} \cdot \begin{bmatrix} 0.596 \\ 0.298 \\ 0.745 \end{bmatrix} = 0.895$$

This number is very close to one, which means that v_1 and v_2 are very similar vectors

Time to Code!

In this chapter, you'll be completing the `compute_cos_sims` function, which computes cosine similarities between vocabulary words. Specifically, you'll be completing the function where it currently has the placeholder "CODE

completing the function where it currently has the placeholder **"CODE HERE"**.

In order for the cosine similarities to be between 0 and 1, we need to first normalize both our retrieved embedding vector and the embedding matrix.

Set `normalized_embedding` equal to `tf.nn.l2_normalize` applied with `word_embedding` as the only argument.

Set `normalized_matrix` equal to `tf.nn.l2_normalize` applied with `self.embedding_matrix` as the required argument and `axis=1` as the keyword argument.

We can now calculate the embedding vector cosine similarities by matrix multiplying `normalized_embedding` and `normalized_matrix`. The matrix multiplication returns a vector with shape `(1, vocab_size)`, where each index contains a cosine similarity between the embedding vector for `word` and the embedding vector for the vocabulary word whose ID matches the index.

Set `cos_sims` equal to `tf.matmul` applied with `normalized_embedding` and `normalized_matrix` as the required arguments, and `transpose_b=True` as the keyword argument. Then return `cos_sims`.

```
import tensorflow as tf

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Forward run of the embedding model to retrieve embeddings
    def forward(self, target_ids):
        initial_bounds = 0.5 / self.embedding_dim
        initializer = tf.random_uniform(
            [self.vocab_size, self.embedding_dim],
            minval=-initial_bounds,
            maxval=initial_bounds)
        self.embedding_matrix = tf.get_variable('embedding_matrix',
            initializer=initializer)
        embeddings = tf.nn.embedding_lookup(self.embedding_matrix, target_ids)
        return embeddings

    # Compute cosine similarites between the word's embedding
    # and all other embeddings for each vocabulary word
    def compute_cos_sims(self, word, training_texts):
        self.tokenizer.fit_on_texts(training_texts)
        word_id = self.tokenizer.word_index[word]
        word_embedding = self.forward([word_id])
```

```
        word_embedding = self.forward([word_id])
        # CODE HERE
```

K-Nearest Neighbors

In this chapter, we will be learning how to calculate K-nearest neighbors based on cosine similarity.

Chapter Goals:

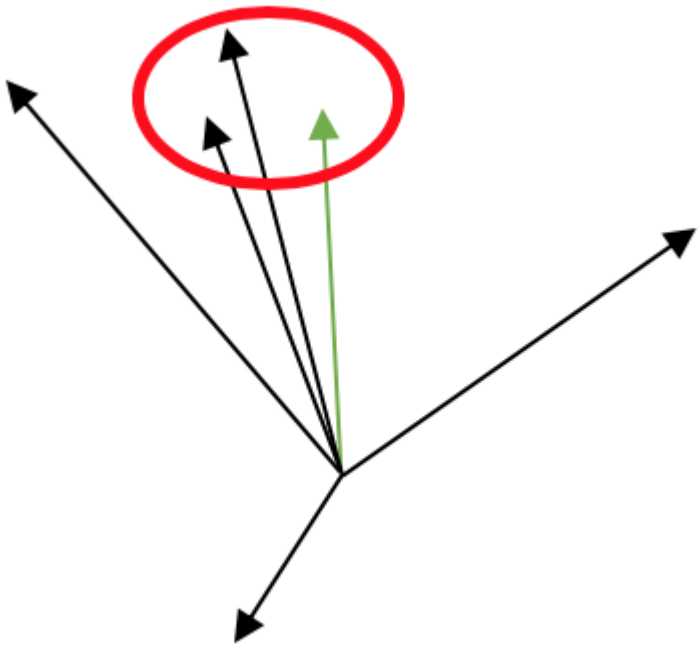
- Learn about K-nearest neighbors in terms of word similarity
- Create a function that computes the K-nearest neighbors for a given word

A. Similar words

When comparing cosine similarities for word embeddings, a common procedure is to find the *K-nearest neighbors* for a given word. This means that for a given a word w and an integer K, we can find the K vocabulary words whose embedding vectors have the highest cosine similarity to the embedding vector for w.

Using K-nearest neighbors can help us evaluate our embedding model and make sure it was trained properly. For example, if we notice that the K-nearest neighbors for any given word is always the same K words, then something may have gone wrong with our embedding training. Likewise, if the K-nearest neighbors are completely different from what we would expect to see (e.g. if the 3 nearest neighbors for the word "computer" are "waterfall", "ocean", and "soda"), then we may want to take a closer look at our model's embedding matrix.

When the embedding model is trained well, the K-nearest neighbors metric can provide some useful insights about the text corpus, particularly for specialized text corpora. For example, we normally expect the word "code" to be related to terms about computer programming or software. However, if our training corpus were related to military operations, the K-nearest neighbors for "code" might include words like "signal", "transmission", or "decode".



Circled in red are the K=2 nearest neighbors of the green vector. The nearest neighbors have the highest cosine similarity, meaning the directions they point in are nearest the green vector's direction

Time to Code!

In this chapter, you'll be completing the `k_nearest_neighbors` function, which computes the K-nearest neighbors for an input `word` using the TensorFlow utility function `tf.math.top_k`.

To find the K-nearest neighbors for `word`, we need to compute the cosine similarities between the embedding vectors for `word` and every other vocabulary word.

Set `cos_sims` equal to `self.compute_cos_sims` applied with `word` and `training_texts` as the arguments.

The returned `cos_sims` has shape `(1, self.vocab_size)`. However, when calculating the K-nearest neighbors, the extra dimension of size `1` is unnecessary. We can remove it using `tf.squeeze`.

Set `squeezed_cos_sims` equal to `tf.squeeze` applied with `cos_sims` as the only argument.

Now we can retrieve the K-nearest neighbors for `word`. The specific number of neighbors we retrieve is given by the integer argument, `k`.

Set `top_k_output` equal to `tf.math.top_k` applied with `squeezed_cos_sims` and `k` as the two arguments. Then return `top_k_output`.

```
import tensorflow as tf

# Skip-gram embedding model
class EmbeddingModel(object):
    # Model Initialization
    def __init__(self, vocab_size, embedding_dim):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Forward run of the embedding model to retrieve embeddings
    def forward(self, target_ids):
        initial_bounds = 0.5 / self.embedding_dim
        initializer = tf.random_uniform(
            [self.vocab_size, self.embedding_dim],
            minval=-initial_bounds,
            maxval=initial_bounds)
        self.embedding_matrix = tf.get_variable('embedding_matrix',
            initializer=initializer)
        embeddings = tf.nn.embedding_lookup(self.embedding_matrix, target_ids)
        return embeddings

    # Compute cosine similarites between the word's embedding
    # and all other embeddings for each vocabulary word
    def compute_cos_sims(self, word, training_texts):
        self.tokenizer.fit_on_texts(training_texts)
        word_id = self.tokenizer.word_index[word]
        word_embedding = self.forward([word_id])
        normalized_embedding = tf.nn.l2_normalize(word_embedding)
        normalized_matrix = tf.nn.l2_normalize(self.embedding_matrix, axis=1)
        cos_sims = tf.matmul(normalized_embedding, normalized_matrix,
            transpose_b=True)
        return cos_sims

    # Compute K-nearest neighbors for input word
    def k_nearest_neighbors(self, word, k, training_texts):
        # CODE HERE
        pass
```

Note that the output `top_k_output` is a tuple. The first element is the top K cosine similarities, while the second element is the actual word IDs corresponding to the top K nearest neighbors.

Quiz

1

Why do we use word embeddings?

COMPLETED 0%

1 of 4

Introduction

In this section you will be building an LSTM (long short-term memory) network for language modeling.

A. Language modeling

The main idea of machine learning in NLP is to train a model to understand a text corpus well enough so that it can automatically perform tasks such as text classification or text generation. In order to do this, a machine learning model must be able to quantify what counts as a "good" or "bad" sequence of words.

Like every machine learning task, the task of quantifying text quality is centered around calculating probabilities. In this case, to calculate how "good" a sentence is, we would need to calculate how "good" each word in the sentence is. We can perform this task using a *language model*.

A language model can tell us the likelihood of each word in a given sentence or text passage based on the words that came before it. We can then determine how likely a sentence or text passage is by aggregating its individual word probabilities.

B. Language model tasks

Language models are useful for both text classification and generation. In text classification, we can use the language model's probability calculations to separate texts into different categories. For example, if we trained a language model on spam email subject titles, the model would likely give the subject "CLICK HERE FOR FREE EASY MONEY" a relatively high probability of being spam.

In text generation, a language model completes a sentence by generating text based on the incomplete input sentence. This is the idea behind the autocomplete feature when texting on a phone or typing in a search engine. The model will give suggestions to complete the sentence based on the words

The model will give suggestions to complete the sentence based on the words it predicts with the highest probabilities.

Language Model

Learn how language models are trained to calculate word probabilities.

Chapter Goals:

- Understand how a language model works
- Learn how to set up the training data for a language model

A. Word probabilities

As mentioned in the introduction, the purpose of a language model is to assign probabilities to words in sequences of text. The probability for each word is conditioned on the words that appear before it in the sequence. Though it may not seem like it at first glance, the task of calculating a word probability based on the previous words in a sequence is essentially multiclass classification.

Since each word in a sequence must come from the text corpus' vocabulary, we consider each vocabulary word as a separate class. We then use the previous sequence words as input into our language model to calculate probabilities for each vocabulary class.

he went to the _____

Vocabulary	Probability
he	0.02
went	0.02
to	0.01
the	0.0
coffee	0.15
store	0.75
wrist	0.05

Multiclass probabilities over the 7 possible vocabulary classes. In this case, the sequence ["he", "went", "to", "the"] is the input to the language model.

B. Inputs & targets

The main idea behind training a language model is to predict each word in a sequence based on the words that come before it. However, rather than making training pairs where the target is a single word, we instead make training pairs consisting of equal length input and target sequences.

Original Sequence: ["she", "bought", "a", "book", "from", "me"]

Input Sequence: ["she", "bought", "a", "book", "from"]

Target Sequence: ["bought", "a", "book", "from", "me"]

Example training pair from a corpus text sequence. Notice that the target sequence is just the input sequence shifted to the right by one word.

The language model attempts to correctly predict each word in the target sequence based on its corresponding prefix in the input sequence. In the example above, the input prefix-target word pairs are:

- (["she"], "bought")
- (["she", "bought"], "a")
- (["she", "bought", "a"], "book")
- (["she", "bought", "a", "book"], "from")
- (["she", "bought", "a", "book", "from"], "me")

For each pair, we'd ideally want the language model to calculate a very high probability for the target word based on the input prefix. This is the same goal as training a multiclass classification model.

C. Maximum length

When creating the input-target sequences for training a language model, a big factor to consider is the length of the sequences. Depending on the text corpus, we can choose to limit our training pair sequences to a fixed maximum length. Using a fixed max sequence length can increase the training speed and also help the language model avoid overfitting uncommon text dependencies (which are sometimes found in long, run-on sentences).

Original Sequence: ["they", "went", "to", "the", "big", "red", "truck"]

Input Sequence: ["they", "went", "to"]

Target Sequence: ["went", "to", "the"]

Example training pair with a fixed max length of 4. Notice that only the first 4 words from the original sequence are considered for the input-target pair.

Time to Code!

In this section of the course, you'll be creating a `LanguageModel` object.

Specifically in this chapter, you'll be completing a helper function for the `get_input_target_sequence` function, which converts a sequence into an input-target pair.

The function you'll complete is the `truncate_sequences` function. This function is used when the length of `sequence` is not less than `self.max_length`.

In this case, we focus on the prefix of `sequence` with length `self.max_length`. The input sequence will be equal to that prefix minus the final token, while the target sequence will be equal to the prefix minus the first token.

Inside the `truncate_sequences` function, set `input_sequence` equal to `sequence` containing every token up to `max_length - 1` (exclusive).

Inside the `truncate_sequences` function, set `target_sequence` equal to `sequence` containing every token from index `1` (inclusive) up to `max_length` (exclusive).

Return a tuple with `input_sequence` as the first element and `target_sequence` as the second element.

```
import tensorflow as tf

def truncate_sequences(sequence, max_length):
    # CODE HERE
    pass

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    def get_input_target_sequence(self, sequence):
        seq_len = len(sequence)
        if seq_len >= self.max_length:
            input_sequence, target_sequence = truncate_sequences(
                sequence, self.max_length
            )
        else:
            # Next chapter
```

```
            input_sequence, target_sequence = pad_sequences(
                sequence, self.max_length


            )
        return input_sequence, target_sequence
```

Padding

Understand the purpose of padding with respect to tokenized sequences.

Chapter Goals:

- Learn about sequence lengths and padding

A. Varied length sequence

When dealing with most neural networks, the input data always has a fixed length. This is because most neural networks have what's known as a *feed-forward* structure, meaning that they utilize multiple layers of fixed sizes to compute the network's output.

However, since text data involves different sized text sequences (e.g. sentences, passages, etc.), our language model needs to be able to handle input data of varied lengths. Therefore, we use a *recurrent* neural network (discussed in greater detail in upcoming chapters) for the basis of our language model.

B. Padded sequences

While the recurrent neural network architecture allows the language model to take in different length input texts, we still need each tokenized text sequence in a training batch to be the same length. This is because a training batch must have proper tensor shape, which in this case means it needs to be a 2-D matrix.

The way we mimic a proper 2-D matrix shape with sequences of varied lengths is to apply *padding*. For each sequence that's shorter than the maximum sequence length, we append a special non-vocabulary token to the end of the sequence until its length is equal to the maximum sequence length. Usually the special padding token is given the ID 0, while each vocabulary word ID is a positive integer.

$$[1, 5, 10, 9, 0, 0, 0]$$

$$[1, 5, 10, 9, 0, 0, 0]$$

$$[12, 2, 3, 4, 17, 8, 2]$$
$$[3, 6, 1, 0, 0, 0, 0]$$

$$[9, 14, 9, 16, 17, 0, 0]$$

A batch of 4 input sequences for a language model. The maximum sequence length is 7, so each sequence is padded with 0's until its length is 7.

Time to Code!

In this chapter, you'll be completing another helper function for the `get_input_target_sequence` function.

The helper function you'll complete is the `pad_sequences` function. This function is used when the length of `sequence` is less than `self.max_length`.

When the length of `sequence` is less than `self.max_length`, we need to add padding. The amount of padding is just the difference between the maximum sequence length and the length of `sequence`.

Inside the `pad_sequences` function, set `padding_amount` equal to `max_length` minus the length of `sequence`.

The `Tokenizer` object never uses `0` as an ID corresponding to a corpus word. This helps us a lot, since we can now safely use `0` as our padding token without worrying about conflicts.

Set `padding` equal to a list of all `0`, whose length is `padding_amount`.

As in the previous chapter, `input_sequence` represents the input sequence with the final token removed, while `target_sequence` represents the target sequence with the first token removed.

The difference here is that both the input and target sequences will be padded up to `max_length`.

Set `input_sequence` equal to `sequence[:-1]` with `padding` added to the end. Also set `target_sequence` equal to `sequence[1:]` with `padding` added to the end.

Return a tuple with `input_sequence` as the first element and `target_sequence` as the second element.

```
import tensorflow as tf

def pad_sequences(sequence, max_length):
    # CODE HERE
    pass

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    def get_input_target_sequence(self, sequence):
        seq_len = len(sequence)
        if seq_len >= self.max_length:
            input_sequence, target_sequence = truncate_sequences(
                sequence, self.max_length
            )
        else:
            input_sequence, target_sequence = pad_sequences(
                sequence, self.max_length
            )
        return input_sequence, target_sequence
```

RNN/LSTM

Learn about recurrent neural networks and LSTM cells.

Chapter Goals:

- Understand the differences between feed-forward and recurrent neural networks
- Learn about the long short-term memory (LSTM) cell

A. Neural network types

As mentioned in the previous chapter, the MLP is a type of *feed-forward neural network*. This means that each of its layers is a fixed size, and each layer's output is fed into the next layer as input. While feed-forward neural networks are great for tasks involving fixed size input data, they aren't as great in dealing with sequences of text data.

For the remainder of this course, we'll be focusing on *recurrent neural networks*, which are specially designed to work with sequential data of varying lengths. The main component of a recurrent neural network (RNN) is its *cell*.

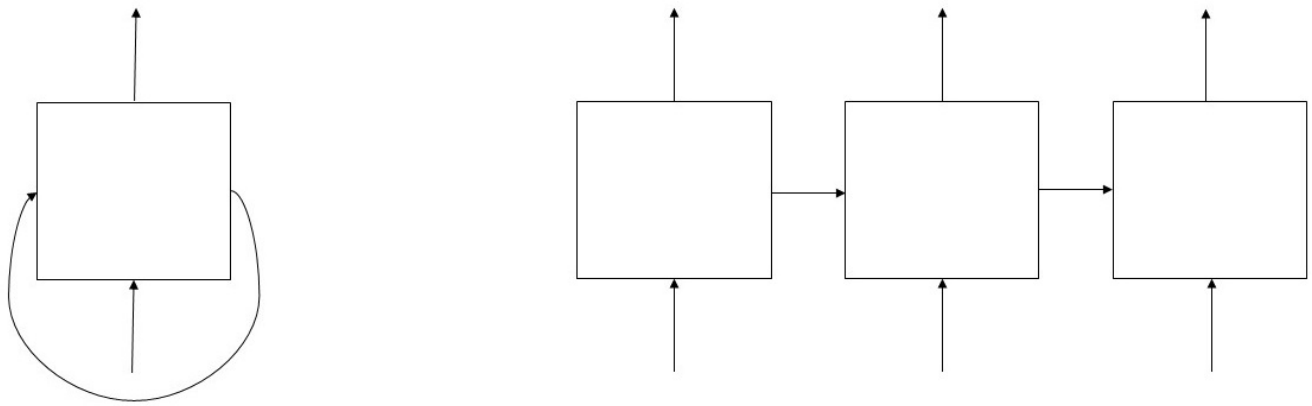


Diagram of an RNN. On the left is the rolled RNN, on the right is the unrolled RNN with 3 time steps. The square represents the RNN cell.

The rolled RNN diagram shows the "true" depiction of the network. It consists of a single cell (although a multi-layer RNN will have multiple stacked cells),

of a single cell (although a multi-layer RNN will have multiple stacked cells),

and 3 types of connections: input, output, and recurrent. The unrolled diagram gives us a better look at what each of these connections represent.

In the unrolled diagram above, the RNN consists of 3 time steps, meaning that the length of the RNN's input sequence is 3. This also means that the RNN will output a sequence of length 3. At each time step, the arrow going into the cell represents the token at that particular index of the input sequence. The arrow coming out of the cell represents the cell's output.

> Note: The diagram shows the RNN process for a single input. For batch data, the RNN will handle each individual input independently, in parallel.

The left-to-right arrows connecting the cell at each time step are the recurrent connections. These are the foundation of an RNN, and they represent the transmission of each time step's *state*. The state gives the cell at the current time step information about the cell inputs and outputs from previous time steps. This is incredibly useful for capturing dependencies in the text that make it easier to calculate probabilities and predictions.

B. LSTM

While theoretically a regular RNN should be able to capture all the necessary dependencies in an input text sequence, this is not usually the case in real life. Specifically, it is difficult for RNNs to handle *long-term* dependencies, i.e. dependencies between words that are far apart in a text sequence.

Let's take a look at the following text sequence:

"She is currently a doctor. She chose this career path because her favorite subject in high school was biology."

Here, "biology" and "doctor" have a dependency and are relatively far from each other in the text sequence. If a human were asked to predict the final word in the text sequence from the previous words, they would likely say "biology", given that the subject is a doctor. However, an RNN would have a difficult time picking up this dependency due to its distance.

In order to handle long-term dependencies, we use a variation of the regular

RNN cell known as the LSTM (long short-term memory). While the default RNN cell has issues with long-term dependencies, the LSTM cell is specifically designed to keep track of all the useful dependencies in a text sequence. . When an RNN uses the LSTM cell variation, we normally just refer to the entire model as an LSTM.

C. Inside the cell

The inside of a default RNN cell consists of two fully-connected layers. The first layer has tanh activation, and it's used to compute the cell's state at a particular time step, based on the previous state and the time step's input.

The second fully-connected layer is unactivated, and it's used to compute the cell's output at the time step. You can think of the number of hidden units in the cell as the number of hidden units in the fully-connected layers.

An LSTM cell adds a few additional layers to the default RNN cell. We can refer to these additional layers as "gates", since they help regulate the information that is added or removed from the cell state. These additional gates give LSTM cells the boost needed to handle long-term dependencies.

Time to Code!

In this chapter, you'll be completing the `make_lstm_cell` function, which creates an LSTM cell with `self.num_lstm_units` as the number of hidden units in the cell.

We use `tf.nn.rnn_cell.LSTMCell` to create the LSTM cell. This function is an initializer for an `LSTMCell` object, and the only required argument is the number of hidden units in the cell.

Side note: For creating a basic RNN cell, you can use the `tf.nn.rnn_cell.BasicRNNCell` function.

Set `cell` equal to `tf.nn.rnn_cell.LSTMCell` applied with `self.num_lstm_units` as the only argument. Then return `cell`.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
```

```
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob):
        # CODE HERE
        pass
```

Dropout

Use dropout to help train a better LSTM model.

Chapter Goals:

- Understand the purpose of dropout in the context of RNNs

Note: While the models used in the remainder of this course are all LSTMs, the concepts from this section of the course are general to any RNN.

A. Regularization

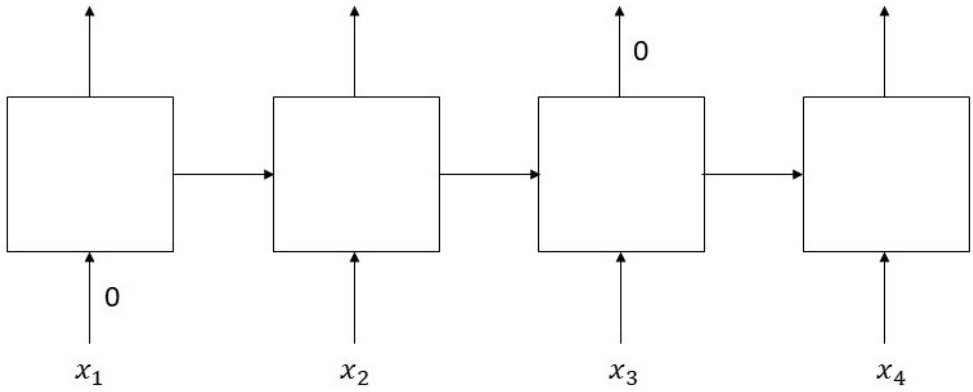
When an RNN has many weight parameters (e.g. the number of hidden units per cell is large or there are multiple RNN layers), it can have a tendency to overfit the training set and therefore generalize poorly to other data. To combat this, we need to *regularize* the RNN during training.

Regularization refers to modifying the neural network in ways that mitigate the risk of overfitting during training. A particularly popular method of regularization for feed-forward neural networks is dropout. Dropout can also be used in recurrent neural networks, although it works slightly differently.

B. Dropout in RNNs

In feed-forward neural networks, dropout refers to randomly "dropping" or zeroing-out certain hidden neurons during training. This helps each neuron in the network become more proficient and less reliant on the other neurons in the network. The fraction of neurons that are randomly dropped out is referred to as the *dropout rate*.

In recurrent neural networks, we apply dropout to the input and/or output of each cell unit. When dropout is applied to a cell's input/output at a particular time step, the cell's connection at that time step is zero'd out. So whatever, the previous input/output value was, it would be set to 0 due to the dropout.



Dropout applied to an RNN with a single cell layer and 4 time steps. Each x_i represents a time step's input. Connections labeled with 0 have dropout applied to them (i.e. zero'd out).

Applying dropout decreases the amount of cell computation that is used in the overall RNN output, thereby reducing the risk that the RNN will overfit the data.

In TensorFlow, the function we use for recurrent neural network dropout is `tf.nn.rnn_cell.DropoutWrapper`, which takes in an RNN/LSTM cell as its required argument. A couple of the more important keyword arguments for the function are `input_keep_prob` and `output_keep_prob`.

The `input_keep_prob` argument represents the probability of *not* dropping (i.e. keeping) the cell's input at each time step. The `output_keep_prob` argument represents the same thing for the cell's outputs. Their default values are both 1.0, which represents no dropout.

When training your model, you can set either argument (or both), depending on how much regularization you want. Similarly, the argument values will also influence the amount of regularization for your model. Usually, a good starting point is around 0.5 (randomly dropping half the inputs/outputs), and then adjusting based on how the model trains.

Time to Code!

In this chapter, you'll be modifying the `make_lstm_cell` function to include dropout.

We can apply (output-only) dropout to the created LSTM cell from the previous chapter. The node keep rate (i.e. 1 minus the dropout rate) is given

previous chapter. The node keep rate (i.e. 1 minus the dropout rate) is given by `dropout_keep_prob`.

Set `dropout_cell` equal to `tf.nn.rnn_cell.DropoutWrapper` applied with `cell` as the required argument and `dropout_keep_prob` as the `output_keep_prob` keyword argument. Then return `dropout_cell`.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob):
        cell = tf.nn.rnn_cell.LSTMCell(
            self.num_lstm_units) # CODE UNDER THIS LINE
```

Multiple Layers

Stack multiple LSTM cell layers for added performance.

Chapter Goals:

- Learn how to stack multiple cell layers in an RNN

A. Stacking layers

Similar to how we can stack hidden layers in an MLP, we can also stack cell layers in an RNN. Adding cell layers allows the model to pick up on more complex features from the input sequence and therefore improve performance when trained on a large enough dataset.

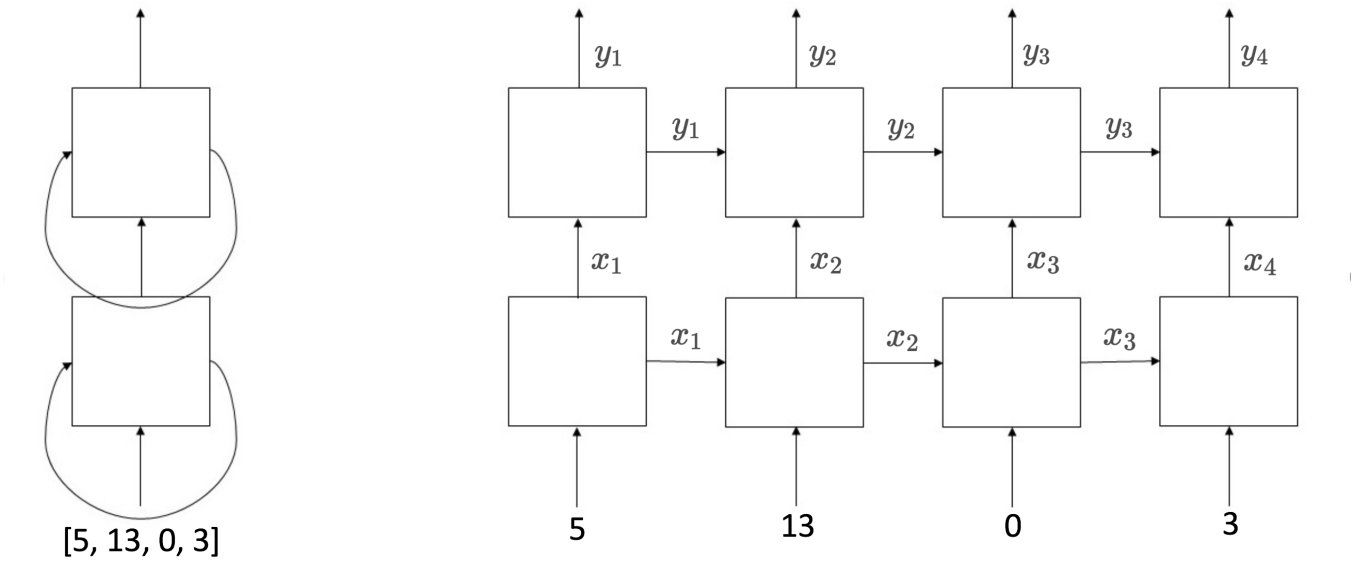


Diagram of an RNN with 2 cell layers. On the left is the rolled RNN, on the right is the unrolled RNN with 4 time steps.

In the diagram above, the RNN contains 2 cell layers (which is just two cells). At each time step, the first cell's output becomes the input for the second cell, and the second cell's output is the overall RNN's output for that particular time step. One thing to note is that the cell states remain separate, i.e. there aren't any recurrent connections between the two cells.

The input in this example is `[5, 13, 0, 3]`, which represents the tokenized sequence for some four word sentence. x_1, x_2, x_3, and x_4 represent the

outputs for the first layer. y_1, y_2, y_3, and y_4 represent the outputs for the second layer.
As with all other neural networks, adding layers can improve performance on larger datasets but also run the risk of overfitting the data. This makes regularization techniques such as dropout more important as we increase the size of our model.

Time to Code!

In this chapter, you'll be modifying the `stacked_lstm_cells` function to create a multi-layer LSTM model.

To create a multi-layer LSTM model, we need to set up a list containing the LSTM cells we want to use in our model. The number of cells will be equal to `self.num_lstm_layers`.

The dropout keep probability for each cell depends on whether we're training the model or not. If the model is training, we'll set the probability to 0.5, otherwise we'll set it to 1.0.

Set `dropout_keep_prob` equal to `0.5` if `is_training` is `True`, otherwise set it to `1.0`.

Create a list called `cell_list`, which contains `self.num_lstm_layers` number of LSTM cells. Each LSTM cell in the list should be the output of a distinct call to `self.make_lstm_cell`, with `dropout_keep_prob` as the only argument.

We can combine each of the LSTM cells into a usable TensorFlow object with the `tf.nn.rnn_cell.MultiRNNCell` function. The function takes in a list of cells as its required argument and returns a `MultiRNNCell` object representing the stacked cells.

Set `cell` equal to `tf.nn.rnn_cell.MultiRNNCell` applied with `cell_list` as the required argument. Then return `cell`.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
```

```
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    # Create a cell for the LSTM
    def make_lstm_cell(self, dropout_keep_prob):
        cell = tf.nn.rnn_cell.LSTMCell(self.num_lstm_units)
        return tf.nn.rnn_cell.DropoutWrapper(
            cell, output_keep_prob=dropout_keep_prob)

    # Stack multiple layers for the LSTM
    def stacked_lstm_cells(self, is_training):
        # CODE HERE
        pass
```

LSTM Output

Run an LSTM model on input sequences and retrieve the output.

Chapter Goals:

- Compute the output of your LSTM model

A. TensorFlow implementation

In TensorFlow, the way we create and run an RNN is with the function `tf.nn.dynamic_rnn`. The function takes in two required arguments. The first is the cell object that is used to create the RNN (e.g. an `LSTMCell`, `MultiRNNCell`, etc.). The second is the batch of input sequences, which are usually first converted to word embedding sequences.

Of the keyword arguments for the function, it is required that either `initial_state` or `dtype` is set. The `initial_state` argument specifies the starting state for the input cell object. We'll use this argument in later parts of this section.

The `dtype` argument specifies the type of both the initial cell state and RNN output. Most of the time, we can just set this argument to `tf.float32`, since the RNN outputs are normally floating point numbers.

Below is an example demonstrating how to use `tf.nn.dynamic_rnn`. Note that the input sequences have maximum length 10 and embedding size 12.

```
import tensorflow as tf
cell = tf.nn.rnn_cell.LSTMCell(7)
# Input sequences for the LSTM
# Shape: (batch_size, time_steps, embed_dim)
input_sequences = tf.placeholder(
    tf.float32,
    shape=(None, 10, 20)
)
output, final_state = tf.nn.dynamic_rnn(
    cell,
    input_sequences,
    dtype=tf.float32
)
```

The `tf.nn.dynamic_rnn` function returns a tuple containing the RNN outputs as well as the final state of the RNN. For now, we only need to focus on the RNN output.

You'll notice from the example that the output first and second dimensions are equal to the input batch. This is because the RNN calculates the output for each time step of each sequence in the input batch. The third dimension, however, is equal to the number of hidden units in the cell object. For RNNs with multiple cells (i.e. `MultiRNNCell` cell object), the third dimension is equal to the number of hidden units in the final cell.

B. Sequence lengths

Because each of the input sequences can have varying lengths, it is likely that many of them will contain padding. Since padding is essentially a sequence filler, and therefore adds nothing of value to the RNN, we don't really want the RNN to waste computation on the padded parts of a sequence.

Instead, we can use the `sequence_length` argument in `tf.nn.dynamic_rnn`. This argument takes in a 1-D integer tensor, specifying the non-padded lengths of each sequence in the input batch.

Below is an example that uses `sequence_length` in `tf.nn.dynmaic_rnn`. The `cell` and `input_sequences` variables are the same as ones from the previous example. In this case, the input batch size is 5.

```
import tensorflow as tf
lens = [4, 9, 10, 5, 10]
cell = tf.nn.rnn_cell.LSTMCell(7)
input_sequences = tf.placeholder(
    tf.float32,
    shape=(None, 10, 20)
)
output, final_state = tf.nn.dynamic_rnn(
    cell,
    input_sequences,
    sequence_length=lens,
    dtype=tf.float32
)
```

By using the `sequence_length` argument, the function can skip unnecessary computation for the padded parts of each sequence, which can greatly reduce training time.

Time to Code!

In this chapter, you'll be completing the `run_lstm` function, which runs the LSTM model on input sequences.

As you can see, the input sequences have already been converted into embeddings, and the sequence lengths have been calculated. What's left for you to do is call the `tf.nn.dynamic_rnn` function to run our LSTM. We'll use `sequence_lengths` for the function's `sequence_length` argument to speed up the computation. We should also set the `dtype` argument to `tf.float32`.

Set `lstm_outputs` equal to the first element of the tuple returned by `tf.nn.dynamic_rnn`, with `cell` and `input_embeddings` as the required arguments to the function. Also use the keyword arguments specified above.

Return a tuple containing `lstm_outputs` as the first element and `binary_sequences` as the second element.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    # Create a cell for the LSTM
    def make_lstm_cell(self, dropout_keep_prob):
        cell = tf.nn.rnn_cell.LSTMCell(self.num_lstm_units)
        return tf.nn.rnn_cell.DropoutWrapper(
            cell, output_keep_prob=dropout_keep_prob)

    # Stack multiple layers for the LSTM
    def stacked_lstm_cells(self, is_training):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob) for i in range(self.num_lstm_laye
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
```

```
        return cell

    # Convert input sequences to embeddings

    def get_input_embeddings(self, input_sequences):
        embedding_dim = int(self.vocab_size**0.25)
        initial_bounds = 0.5 / embedding_dim
        initializer = tf.random_uniform(
            [self.vocab_size, embedding_dim],
            minval=-initial_bounds,
            maxval=initial_bounds)
        self.input_embedding_matrix = tf.get_variable('input_embedding_matrix',
            initializer=initializer)
        input_embeddings = tf.nn.embedding_lookup(self.input_embedding_matrix, input_sequence
        return input_embeddings

    # Run the LSTM on the input sequences
    def run_lstm(self, input_sequences, is_training):
        cell = self.stacked_lstm_cells(is_training)
        input_embeddings = self.get_input_embeddings(input_sequences)
        binary_sequences = tf.sign(input_sequences)
        sequence_lengths = tf.reduce_sum(binary_sequences, axis=1)
        # CODE HERE
```

Calculating Loss

Calculate the loss for your LSTM model.

Chapter Goals:

- Convert your LSTM model's outputs into logits
- Use a padding mask to calculate the overall loss

A. Logits & loss

As mentioned in earlier chapters, the task for a language model is no different from regular multiclass classification. Therefore, the loss function will still be the regular softmax cross entropy loss. We use a final fully-connected layer to convert model outputs into logits for each of the possible classes (i.e. vocabulary words).

```
import tensorflow as tf
# Output from an LSTM
# Shape: (batch_size, time_steps, cell_size)
lstm_outputs = tf.placeholder(tf.float32, shape=(None, 10, 7))

vocab_size = 100
logits = tf.layers.dense(lstm_outputs, vocab_size)

# Target tokenized sequences
# Shape: (batch_size, time_steps)
target_sequences = tf.placeholder(tf.int64, shape=(None, 10))
loss = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=target_sequences,
    logits=logits)
```

Obtaining the loss for an LSTM model (with max sequence length of 5).

The function used to calculate the softmax cross entropy loss for feed-forward neural networks is `tf.nn.softmax_cross_entropy_with_logits`. However, we can only use this function if the `labels` and `logits` arguments both have the same shape.

In our example, `logits` has 3 dimensions while `labels` (`target_sequences`) only has 2. In this case, the `labels` are referred to as *sparse* (i.e. they represent class indexes rather than one-hot vectors), so we use the sparse version of the loss function.

B. Padding mask

When we calculate the loss based on the model's outputs, we don't want to include the logits for every time step in each sequence. Specifically, we want to exclude the loss calculated for the padded time steps, since those values are meaningless. Therefore, we use a *padding mask* to zero-out the loss at padded time steps.

The padding mask will have the same shape as the labels (i.e. target batch), but it will only contain 0's and 1's. Locations containing 0 represent padded time steps while locations containing 1 represent actual input sequence tokens. We multiply the padding mask by the loss to zero-out the padded time step locations.

The code below demonstrates an example usage of a padding mask, with batch size of 1 and max sequence length of 5. Note that we cast the padding mask to `tf.float32` so that it matches the type of the loss.

```
import tensorflow as tf
# loss: Softmax loss for LSTM
with tf.Session() as sess:
    print(repr(sess.run(loss)))

# Same shape as loss
pad_mask = tf.constant([
    [1., 1., 1., 1., 0.],
    [1., 1., 0., 0., 0.]
])

new_loss = loss * pad_mask
with tf.Session() as sess:
    print(repr(sess.run(new_loss)))
```

Time to Code!

In this chapter you'll be completing `calculate_loss` function, which calculates the model loss from the LSTM outputs.

First, we'll convert the outputs of the LSTM model into logits.

Set `logits` equal to `tf.layers.dense` applied with `lstm_outputs` and `self.vocab_size` as the two arguments.

Note that `logits` has shape `(batch_size, self.max_length, self.vocab_size)` while `output_sequences` (the batch of tokenized target sequences) has shape `(batch_size, self.max_length)`. Therefore, we use a sparse softmax cross entropy to compute the loss.

Set `batch_sequence_loss` equal to `tf.nn.sparse_softmax_cross_entropy_loss` applied with `output_sequences` and `logits` for the `labels` and `logits` keyword arguments, respectively.

To zero-out the loss for the padded time steps, we'll use `binary_sequences` (created in the previous chapter) as our padding mask. However, in order to use the padding mask, we need to first cast it to type `tf.float32` (to match the type of the loss).

Cast `binary_sequences` to type `tf.float32` and then multiply it by `batch_sequence_loss`. Store the output in the variable `unpadded_loss`.

The overall loss calculated by the function is the sum of the losses across every time step of each sequence.

Set `overall_loss` equal to `tf.reduce_sum` applied with `unpadded_loss` as the only argument. Then return `overall_loss`.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    # Calculate model loss
    def calculate_loss(self, lstm_outputs, binary_sequences, output_sequences):
        # CODE HERE
        pass
```

Predictions

Create word predictions based on the output of your LSTM model.

Chapter Goals:

- Calculate softmax probabilities to make predictions for each time step

A. Calculating probabilities

When using a feed-forward neural network for multiclass predictions, we first need to convert the model's logits into probabilities with the softmax function. For an RNN language model, the idea is the exact same. After converting the RNN's outputs into logits, we apply the softmax function to the *final dimension* of the logits.

The model we've built so far in this section predicts the likely next word given an input sentence (see the **Language Model** chapter for a refresher)

```
import tensorflow as tf
# Logits with a vocab_size = 100
logits = tf.placeholder(tf.float32, shape=(None, 5, 100))
probabilities = tf.nn.softmax(logits, axis=-1)
```

Using the softmax function to calculate probabilities from logits.

By applying the softmax function to the final dimension of the logits, we calculate probabilities for every vocabulary word at each time step in each sequence. In the example above, the first dimension of `probabilities` corresponds to each sequence in the batch, the second dimension corresponds to the time steps, and the third dimension corresponds to the vocabulary word probabilities.

B. Word predictions

Similar to regular multiclass predictions with an MLP (Multilayer Perceptron)

Similar to regular multiclass predictions with an MLP (Multilayer Perceptron), we calculate the RNN's word predictions by taking the highest probability

word at each time step. The nice thing about how we tokenized our text sequences is that each vocabulary word becomes a positive integer. This means that, for each time step, the index containing the highest probability will be the model's word prediction.

The example below shows how to calculate word predictions from probabilities. We use `tf.argmax` to retrieve the indexes of the final dimension that contain the highest probabilities.

```
import tensorflow as tf
# Placeholder for the model probabilities
probabilities = tf.placeholder(tf.float32, shape=(None, 5, 100))

word_preds = tf.argmax(probabilities, axis=-1)
```

C. Current state of the art

LSTMs are more effective at natural language processing than conventional neural networks, but more effective models do exist.

Currently, the best performing model uses transformers, which are detailed in this paper.

To play around with a model that uses transformers, click this link.

Tensor Indexing

Use tensor indexing to retrieve the model's final word prediction.

Chapter Goals:

- Extract the word predictions for the final time step of each sequence

A. Gather functions

When generating word predictions for a batch of sequences, we only want to retrieve the word IDs for each sequence's final time step. If this were regular Python (or NumPy), we could do this through simple list indexing or slicing. However, in TensorFlow we need to use *gather* functions to retrieve data at specific locations in the tensor.

There are two main gather functions: `tf.gather` and `tf.gather_nd`. Both take in the same two required arguments:

- **params:** The tensor that we wish to retrieve data from.
- **indices:** The locations in the tensor that we will index into.

The `tf.gather` function can be used to retrieve specific slices from a tensor, based on what the `axis` keyword argument is set to (default is `0`).

Below are example usages of `tf.gather`. Note that the values in the `indices` argument must by valid indices for the specified `axis`.

```
import tensorflow as tf
t1 = tf.constant([1, 2, 3])
with tf.Session() as sess:
    print(repr(sess.run(tf.gather(t1, 0))))
    print(repr(sess.run(tf.gather(t1, 2))))

print('\n')
t2 = tf.constant([[1, 2, 3], [4, 5, 6]])
with tf.Session() as sess:
    print(repr(sess.run(tf.gather(t2, 0))))
    print(repr(sess.run(tf.gather(t2, 1, axis=1))))
    print(repr(sess.run(tf.gather(t2, [0, 2], axis=1))))
```

```
print('\n')
t3 = tf.constant([
    [[1, 2, 3], [4, 5, 6]],

    [[5, 6, 7], [7, 8, 9]]
])
with tf.Session() as sess:
    print(repr(sess.run(tf.gather(t3, 0))))
    print(repr(sess.run(tf.gather(t3, 1, axis=1))))
    print(repr(sess.run(tf.gather(t3, [0, 2], axis=2))))
```

While `tf.gather` retrieves slices from the input tensor, we can use `tf.gather_nd` for specific tensor indexing. Since `tf.gather_nd` is used for specific indexing rather than slicing, there's no `axis` keyword argument.

The code below shows example usages of `tf.gather_nd`. Note that the tensors `t2` and `t3` come from the previous example.

```
with tf.Session() as sess:
    print(repr(sess.run(tf.gather_nd(t2, [0, 1]))))
    print(repr(sess.run(tf.gather_nd(t2, [[0, 1], [1, 1]]))))

print('\n')
with tf.Session() as sess:
    print(repr(sess.run(tf.gather_nd(t3, [0, 1]))))
    print(repr(sess.run(tf.gather_nd(t3, [[0, 0], [1, 1]]))))
    print(repr(sess.run(tf.gather_nd(t3, [0, 1, 2]))))
```

When using `tf.gather_nd`, the `params` argument must be a multi-dimensional tensor (cannot be 1-D), and the `indices` argument cannot be a single integer.

Time to Code!

In this chapter you'll be completing the `get_word_predictions` function, which uses tensor indexing to get the final word ID prediction from the full sequence of predicted IDs (`word_preds`).

To retrieve the final time step word predictions from each sequence, we need to create a 2-D matrix with shape `(batch_size, 2)`. Each row of this matrix corresponds to *(x, y)* pair, representing the final time step of a sequence in the batch.

The first column of the 2-D matrix is a range from `0` to `batch_size - 1` (inclusive), corresponding to the row indices of each sequence in the batch. We can use the `tf.range` function to create this range (TensorFlow's version of Python `range`).

Set `row_indices` equal to `tf.range` applied with `batch_size` as the only argument.

The second column of the 2-D matrix contains the final time steps of each sequence. This is equivalent to the length of each sequence subtracted by 1.

Call `tf.reduce_sum` with `binary_sequences` as the required argument and `axis=1` as the keyword argument. Then set `final_indexes` equal to the output of `tf.reduce_sum`, subtracted by 1.

Now that we created both columns of the 2-D matrix, we can put the columns together. Since the list `[row_indices, final_indexes]` corresponds to a matrix with shape `(2, batch_size)`, we can transpose it to get the matrix we want.

Set `gather_indices` equal to `tf.transpose` applied to the specified list.

Using `gather_indices` and the N-dimensional indexing function, `tf.gather_nd`, we can retrieve the final time step word predictions from `word_preds`, which will be the output of the function.

Set `final_id_predictions` equal to `tf.gather_nd` with `word_preds` and `gather_indices` as the two arguments. Then return `final_id_predictions`.

```
import tensorflow as tf

# LSTM Language Model
class LanguageModel(object):
    # Model Initialization
    def __init__(self, vocab_size, max_length, num_lstm_units, num_lstm_layers):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)

    # Predict next word ID
    def get_word_predictions(self, word_preds, binary_sequences, batch_size):
        # CODE HERE
        pass
```

Quiz

1

What is the difference between RNN and LSTM cells?

COMPLETED 0%

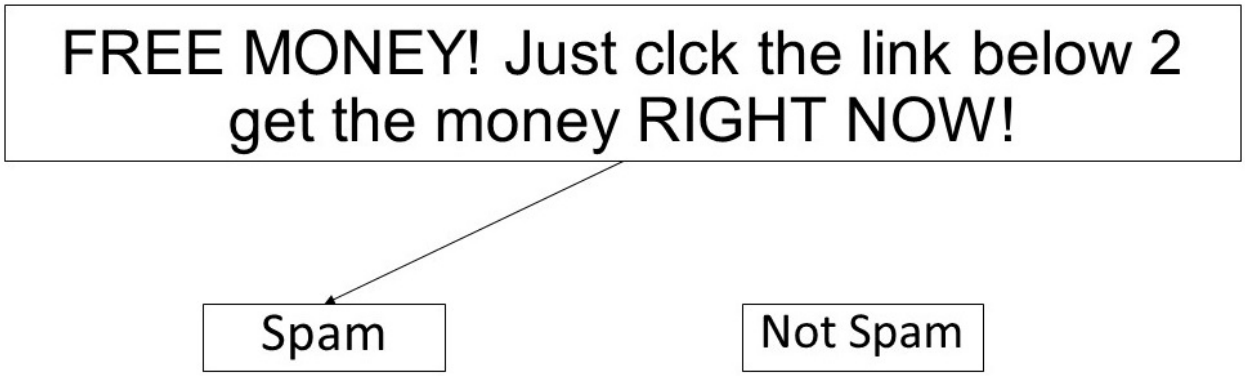
1 of 4

Introduction

In this section you will be building a bidirectional LSTM for text classification. The bidirectional LSTM you build in this section will be trained on a text corpus of movie reviews, separated into positive and negative reviews.

A. Classifying text

One of the most important aspects of NLP is the task of classifying texts into different categories. Applications of text classification include spam filtering, classifying user product reviews, and automatically flagging inappropriate or harmful social media posts.



Text classification in the form of spam filtering. In this example, you see an email get classified as spam.

When dealing with text data it is almost always better to use an LSTM over a regular feed-forward neural network. This is because the LSTM is built specifically for sequential data like text data.

Therefore we'll use a variation known as the bidirectional LSTM for our text classification in the upcoming chapters.

Sentiment Analysis

Learn about sentiment analysis and how it relates to NLP.

Chapter Goals:

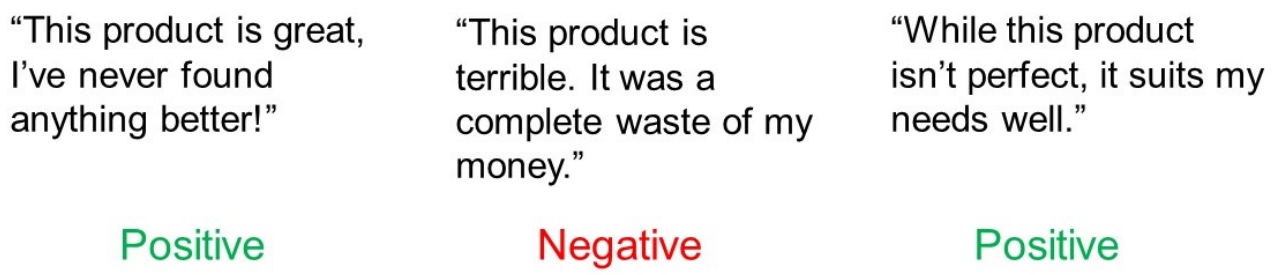
- Learn about sentiment analysis and how it's related to NLP
- Create sentiment analysis training pairs from a text corpus

A. Classifying sentiment

Any time you read an opinionated text passage (e.g. social media post, editorial, etc.), the text contains **sentiment**. Sentiment refers to the writer's attitude in the text, so **sentiment analysis** is the task of analyzing or deducing what the writer's attitude is based on the text.

Oftentimes, the writer's attitude can be classified into one of several categories. For example, online user product reviews are normally given on a 1-5 star scale. In this case, we could separate the reviews based on how many stars they have.

An even simpler classification would be to separate the reviews into two categories: positive and negative. This is just regular binary classification, while the stars example is multiclass classification.



Classifying product reviews as either "Positive" or "Negative".

B. Training pairs

Similar to regular classification, text classification involves training pairs

consisting of input data and labels. In this case, the input data will be tokenized text sequences and each text sequence will be labeled with a class

(category). For simplicity, the class labels are just integers in the range $[0, n-1]$, where n is the total number of classes.

$$([1, 5, 6, 8, 2], 0)$$

$$([3, 5, 2, 9, 8], 1)$$

$$([2, 4, 7, 6, 6], 0)$$

Example batch of 3 training pairs. The maximum sequence length is 5 and there are two classes (binary classification).

Time to Code!

In this section of the course you'll be creating a general text classification model, by completing the `ClassificationModel` object.

Specifically, in this chapter you'll be completing the `make_training_pairs` function, which creates training pairs for the classification model.

The function has already been filled with code that processes raw text into tokenized, truncated sequences. Your job is to combine the training sequences and labels into training pairs.

Set `training_pairs` equal to `zip(sequences, labels)`, cast as a `list`. Then return `training_pairs`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column

# Text classification model
class ClassificationModel(object):
    # Model initialization
    def __init__(self, vocab_size, max_length, num_lstm_units):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    def tokenize_text_corpus(self, texts):
        self.tokenizer.fit_on_texts(texts)
        sequences = self.tokenizer.texts_to_sequences(texts)
        return sequences

    # Create training pairs for text classification
    def make_training_pairs(self, texts, labels):
        sequences = self.tokenize_text_corpus(texts)
```

```
        sequences = self.tokenize_text_corpus(texts)
        for i in range(len(sequences)):
            sequence = sequences[i]
            if len(sequence) > self.max_length:
                sequences[i] = sequence[:self.max_length]
        # CODE HERE
```

Embeddings

Use TensorFlow's built-in functions for creating embeddings.

Chapter Goals:

- Use TensorFlow feature columns to create input embeddings

A. Feature columns

So far, we've dealt with pre-training an embedding model as well as training an embedding model in tandem with an LSTM. In both of these cases, we had to write our own embedding matrix and handle its initialization.

A third option for incorporating word embeddings into your LSTM training model is by using TensorFlow's feature column API. Most of these feature columns won't be discussed in this course, since they aren't directly relevant to NLP.

The feature column function we will focus on is `tf.feature_column.embedding_column`. It lets you incorporate an embedding matrix automatically into your model, which will be trained alongside the LSTM.

B. Sequential categorical column

The `embedding_column` function takes in two required arguments. The second required argument is just the embedding size, which we normally set to the 4th root of the vocabulary size. The first required argument is something called a categorical column.

The feature columns API provides functions for creating categorical columns, but none of these functions work for sequential data (i.e. data with time steps). For functions that work with sequential data, we need to use the extended feature column API, found in `tf.contrib.feature_column`.

Note: Any code found in the `tf.contrib` submodule is volatile, meaning that it may be moved to the main TensorFlow module in future updates.

the main TensorFlow module in future updates.

There are four functions for creating categorical columns with sequential data:

- `sequence_categorical_column_with_identity`
- `sequence_categorical_column_with_hash_bucket`
- `sequence_categorical_column_with_vocabulary_file`
- `sequence_categorical_column_with_vocabulary_list`

Since each of the vocabulary words in the text corpus are converted to unique integer IDs, we only need to focus on the first function. The other three functions all involve some form of text sequence processing, such as hashing words into IDs or using a specified vocabulary list/file.

The categorical column itself is a placeholder for a categorical feature. Categorical columns don't contain any data until the computation graph is run, similar to `tf.placeholder` objects.

The `sequence_categorical_column_with_identity` function takes in two required arguments: a string name for the categorical column and the vocabulary size of the text corpus. The string name will be used when creating the input dictionary for the main conversion function.

The example below creates an embedding column (`embed_col`) using a categorical column (`input_col`) as input.

```
import tensorflow as tf
vocab_size = 10000
input_col = tf.contrib.feature_column \
               .sequence_categorical_column_with_identity(
                   'input', vocab_size)
embed_size = int(10000**0.25)
embed_col = tf.feature_column.embedding_column(
                   input_col, embed_size)
```

C. Converting to embeddings

The main conversion function used to create the embedded input sequences is `sequence_input_layer`. Like the other functions, it takes in two required

arguments as well. The first is a dictionary of input data, where each key is

the name of a categorical column. The second is a list of feature columns corresponding to data in the input dictionary.

In our case, since the input data is just a batch of tokenized sequences, the dictionary will only contain a single key-value pair. The key will be the same string we set in the `sequence_categorical_column_with_identity` function. Furthermore, the second argument will just be a list containing the embedding column.

Below, we show an example using `sequence_input_layer` with tokenized sequences (`input_seqs`) and `embed_col` from the previous example.

```
import tensorflow as tf
# Input batch of tokenized sequences (30 time steps)
input_seqs = tf.placeholder(tf.int64, shape=(None, 30))

input_dict = {'input': input_seqs}
embed_seqs, sequence_lengths = tf.contrib.feature_column \
                                  .sequence_input_layer(
                                      input_dict, [embed_col])
```

The output of `sequence_input_layer` is a tuple containing the embedded sequences and the sequence lengths. The sequence lengths output is used for calculating the sequence lengths for variable length sequence inputs.

Bidirectional LSTM

Create and run a bidirectional LSTM model.

Chapter Goals:

- Learn about the bidirectional LSTM and why it's used

A. Forwards and backwards

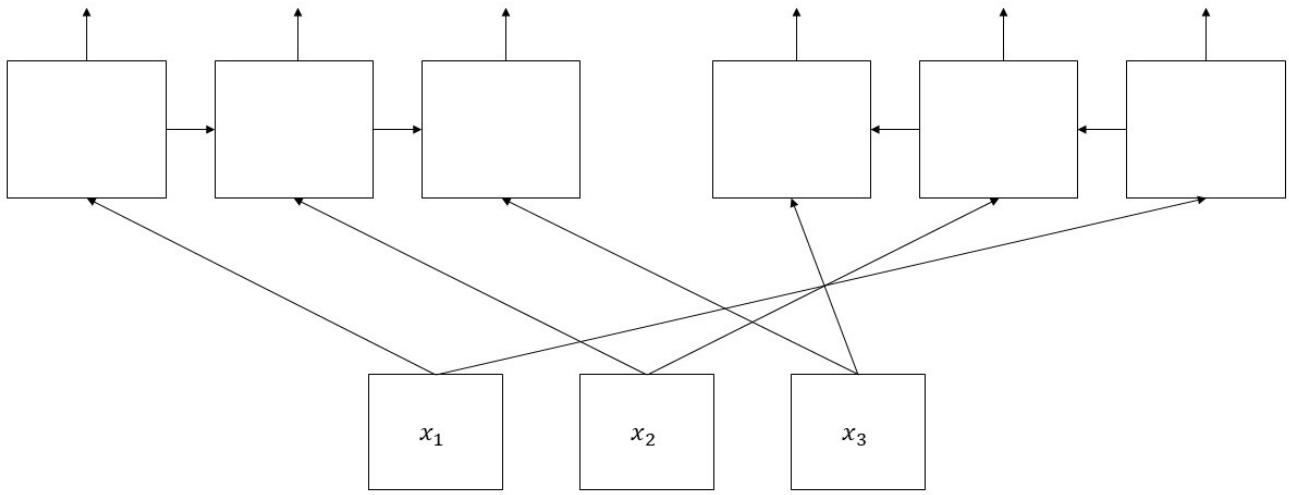
The language model from the **Language Model** section of this course used a regular LSTM which read each input sequence in the *forwards* direction. This meant that the recurrent connections went in the left-right direction, i.e. from time step t to time step $t + 1$.

While regular LSTMs work well for most NLP tasks, they are not always the best option. Specifically, when we have access to a completed text sequence (e.g. text classification), it may be beneficial to look at the sequence in both the forwards and *backwards* directions.

By looking at a sequence in both directions, the model can take into account both the past and future context of words and phrases in the text sequence, which allows it to obtain a better understanding of the text sequence.

B. Bidirectional LSTM structure

The way we look at an input sequence in both directions is with a model called a *bidirectional LSTM*, or BiLSTM for short. The model architecture is incredibly simple: it consists of a regular forwards LSTM and a backwards LSTM, which reads the input sequence in reverse.



The above diagram shows a (unrolled) BiLSTM with 3 time steps. On the left is the forwards LSTM and on the right is the backwards LSTM. The sequence $[x_1, x_2, x_3]$ represents an input (embedded) sequence.

Note: It is also possible to create a bidirectional RNN with general RNN cells rather than LSTM cells. However, since this Lab focuses on LSTM cells, we'll continue using the BiLSTM variant.

C. BiLSTM in TensorFlow

In TensorFlow, we can create and run a BiLSTM using the `tf.nn.bidirectional_dynamic_rnn` function. This function is very similar to the `tf.nn.dynamic_rnn` function for regular LSTMs, with the main difference being that it takes in two LSTM cells rather than one.

The code below shows example usage of `tf.nn.bidirectional_dynamic_rnn`. The `cell_fw` and `cell_bw` variables represent the forwards and backwards LSTM cells, respectively.

```
import tensorflow as tf
cell_fw = tf.nn.rnn_cell.LSTMCell(7)
cell_bw = tf.nn.rnn_cell.LSTMCell(7)

# Embedded input sequences
# Shape: (batch_size, time_steps, embed_dim)
input_embeddings = tf.placeholder(
    tf.float32, shape=(None, 10, 12))
outputs, final_states = tf.nn.bidirectional_dynamic_rnn(
    cell_fw,
    cell_bw,
    input_embeddings
```

```
    dtype=tf.float32)
print(outputs[0])
print(outputs[1])
```

The `tf.nn.bidirectional_dynamic_rnn` function returns a tuple containing the LSTM outputs and the final LSTM states. Since a BiLSTM contains two LSTMs, both `outputs` and `final_states` shown in the example are tuples. We won't worry about `final_states` for now.

However, note that `outputs[0]` represents the outputs of the forwards LSTM while `outputs[1]` represents the outputs of the backwards LSTM. This is important for calculating the model's logits (which we'll do in the next chapter).

Time to Code!

In this chapter you'll be completing the `run_bilstm` function, which runs a bidirectional LSTM on input sequences.

The function has already been filled with code that converts the sequences to embeddings and uses the `make_lstm_cell` function to create the directional LSTM cells. Your task is to use the `tf.nn.bidirectional_dynamic_rnn` function to run the BiLSTM.

We only need to use the first element of the returned tuple from running the BiLSTM. We also use `sequence_lengths` and `tf.float32` for the `sequence_length` and `dtype` keyword arguments, respectively.

Set `lstm_outputs` equal to the first element of the tuple returned by `tf.nn.bidirectional_dynamic_rnn`. Use `cell_fw`, `cell_bw`, and `input_embeddings` as the required arguments and also use the keyword arguments specified above.

Return a tuple containing `lstm_outputs` as the first element and `sequence_lengths` as the second element.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column

# Text classification model
```

```
# Text Classification Model
class ClassificationModel(object):
    # Model initialization
    def __init__(self, vocab_size, max_length, num_lstm_units):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Make LSTM cell with dropout
    def make_lstm_cell(self, dropout_keep_prob):
        cell = tf.nn.rnn_cell.LSTMCell(self.num_lstm_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    # Use feature columns to create input embeddings
    def get_input_embeddings(self, input_sequences):
        inputs_column = tf_fc.sequence_categorical_column_with_identity(
            'inputs',
            self.vocab_size)
        embedding_column = tf.feature_column.embedding_column(
            inputs_column,
            int(self.vocab_size**0.25))
        inputs_dict = {'inputs': input_sequences}
        input_embeddings, sequence_lengths = tf_fc.sequence_input_layer(
            inputs_dict,
            [embedding_column])
        return input_embeddings, sequence_lengths

    # Create and run a BiLSTM on the input sequences
    def run_bilstm(self, input_sequences, is_training):
        input_embeddings, sequence_lengths = self.get_input_embeddings(input_sequences)
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_fw = self.make_lstm_cell(dropout_keep_prob)
        cell_bw = self.make_lstm_cell(dropout_keep_prob)
        # CODE HERE
```

Logits

Calculate logits based on the final output of the model.

Chapter Goals:

- Calculate logits from the combined BiLSTM outputs

A. Concatenation

As mentioned in the previous chapter, the BiLSTM returns two outputs: the forwards and backwards outputs. In order to calculate the model's logits, we need to combine these two outputs. We do this through simple *concatenation*.

Concatenation in TensorFlow refers to appending tensors along a certain dimension. The function that performs this operation is `tf.concat`. It takes in two required arguments: a list of tensors to concatenate and the axis (dimension) to concatenate along.

Below we demonstrate an example usage of `tf.concat`. The variables `o1` and `o2` are NumPy arrays representing the concatenation outputs.

```
import tensorflow as tf
# Shape: (2, 2, 3)
t1 = tf.constant([
    [[1, 2, 3], [4, 5, 6]],
    [[0, 4, 8], [3, 2, 2]]
])

# Shape: (1, 2, 3)
t2 = tf.constant([
    [[9, 9, 9], [8, 8, 8]]
])

# Shape: (2, 2, 2)
t3 = tf.constant([
    [[9, 9], [1, 1]],
    [[7, 2], [8, 8]]
])

with tf.Session() as sess:
    o1 = sess.run(tf.concat([t1, t2], 0))
    o2 = sess.run(tf.concat([t1, t3], -1))
```

```
print(repr(o1))
print(repr(o2))
```

When concatenating tensors, each tensor needs to have the exact same shape, apart from the axis that's being concatenated. The tensors are concatenated in the same order that they appear in the list.

We can use `-1` for the second argument to specify the final tensor dimension as the axis of concatenation. This is a useful shortcut for concatenating along the final dimension, and it is how we combine the BiLSTM outputs.

B. Final time step

Unlike the language model from the **Language Model** section of this course, when we create an LSTM for classification we only use the final time step output for each sequence in the batch. This is because we take into account the entire text sequence for classification, whereas for language modeling we were focused on completing partial sequences.

So after combining the forwards and backwards LSTM outputs, we retrieve the final time step values (using `tf.gather_nd`), and then pass those values through a final fully-connected layer to obtain the model's logits.

Time to Code!

In this chapter you'll be completing the `calculate_logits` function, which calculates logits based on the outputs of the BiLSTM.

The function's input, `lstm_outputs`, is a tuple containing the outputs of the forwards and backwards LSTMs. Our first step is to separate the tuple into two distinct variables.

Set `lstm_outputs_fw, lstm_outputs_bw` equal to `lstm_outputs`.

The way we combine the two LSTM outputs is by concatenating the output values along their final dimension. The input list for `tf.concat` should be `[lstm_outputs_fw, lstm_outputs_bw]`.

Set `combined_outputs` equal to `tf.concat` applied with the specified input list as the first argument and `-1` as the second argument.

list as the first argument and `1` as the second argument.

We provide a function, `get_gather_indices`, which uses code from the **Language Model** section to calculate the indices of each sequence's final time step. Use that function, along with `tf.gather_nd`, to retrieve the final time step values from `combined_outputs`.

Set `gather_indices` equal to `self.get_gather_indices` applied with `batch_size` and `sequence_lengths` as arguments.

Set `final_outputs` equal to `tf.gather_nd` applied with `combined_outputs` and `gather_indices` as arguments.

Since our task is binary text classification, we use a final fully-connected layer with a single node to obtain the model's logits.

Set `logits` equal to `tf.layers.dense` applied with `final_outputs` as the first argument and `1` as the second argument. Then return `logits`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column

# Text classification model
class ClassificationModel(object):
    # Model initialization
    def __init__(self, vocab_size, max_length, num_lstm_units):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    def get_gather_indices(self, batch_size, sequence_lengths):
        row_indices = tf.range(batch_size)
        final_indexes = tf.cast(sequence_lengths - 1, tf.int32)
        return tf.transpose([row_indices, final_indexes])

    # Calculate logits based on the outputs of the BiLSTM
    def calculate_logits(self, lstm_outputs, batch_size, sequence_lengths):
        # CODE HERE
        pass
```

Loss

Calculate the model's sigmoid cross entropy loss.

Chapter Goals:

- Calculate the model's loss using sigmoid cross entropy

A. Sigmoid cross entropy

The task for our model is to classify input text sequences as either negative (label `0`) or positive (label `1`). This is equivalent to binary classification. As with regular binary classification, we use sigmoid cross entropy to calculate the model's loss.

For an in-depth and intuitive explanation of sigmoid cross entropy and binary classification, check out the Machine Learning for Software Engineers course on Educative.

Time to Code!

In this chapter you'll be completing the `calculate_loss` function, which calculates model loss based on the outputs of the BiLSTM.

The first step to calculating the model's loss is to first calculate the logits. We can use the `calculate_logits` function we completed in the previous chapter.

Set `logits` equal to `self.calculate_logits` applied with `lstm_outputs`, `batch_size`, and `sequence_lengths` as arguments.

Since we're performing binary classification, we use sigmoid cross entropy for the loss. We also need to convert the integer labels into floats.

Set `float_labels` equal to `tf.cast` applied with `labels` as the first argument and `tf.float32` as the second argument.

Set `batch_loss` equal to `tf.nn.sigmoid_cross_entropy_with_logits` applied with `float_labels` and `logits` for the `labels` and `logits` keyword

arguments, respectively.

The output of the function is the overall aggregate loss. This means we need to sum together each individual sequence's loss in the batch.

Set `overall_loss` equal to `tf.reduce_sum` applied to `batch_loss`. Then return `overall_loss`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column

# Text classification model
class ClassificationModel(object):
    # Model initialization
    def __init__(self, vocab_size, max_length, num_lstm_units):
        self.vocab_size = vocab_size
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    def get_gather_indices(self, batch_size, sequence_lengths):
        row_indices = tf.range(batch_size)
        final_indexes = tf.cast(sequence_lengths - 1, tf.int32)
        return tf.transpose([row_indices, final_indexes])

    # Calculate logits based on the outputs of the BiLSTM
    def calculate_logits(self, lstm_outputs, batch_size, sequence_lengths):
        lstm_outputs_fw, lstm_outputs_bw = lstm_outputs
        combined_outputs = tf.concat([lstm_outputs_fw, lstm_outputs_bw], -1)
        gather_indices = self.get_gather_indices(batch_size, sequence_lengths)
        final_outputs = tf.gather_nd(combined_outputs, gather_indices)
        logits = tf.layers.dense(final_outputs, 1)
        return logits

    # Calculate the loss for the BiLSTM
    def calculate_loss(self, lstm_outputs, batch_size, sequence_lengths, labels):
        # CODE HERE
        pass
```

Classification

Use the model to classify a given text sequence.

Chapter Goals:

- Classify input text using the BiLSTM model

A. Calculating predictions

our BiLSTM is used to classify sentiment in text as either positive or negative. Since there are only two classes, this is binary classification. For binary classification, we apply the sigmoid function on the model's logits to obtain a probability for the input text. The probability refers to the likelihood that the input text is labeled "positive" (i.e. label `1`), and we can simply round the probability to the nearest integer to obtain the model's prediction.

Time to Code!

In this chapter you'll be completing the `logits_to_predictions` function, which converts the model's logits to binary classification predictions.

To calculate the model's prediction, we first need to obtain probabilities from the logits. We do this by applying the sigmoid function.

Set `probs` equal to `tf.nn.sigmoid` applied to `logits`.

Our model's predictions now become the probabilities rounded to the nearest integer.

Set `preds` equal to `tf.round` applied to `probs`. Then return `preds`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column

# Text classification model
class ClassificationModel(object):
    # Model initialization
    def __init__(self, vocab_size, max_length, num_lstm_units):
        self.vocab_size = vocab_size
```

```
        self.max_length = max_length
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=self.vocab_size)

    # Convert logits to predictions
    def logits_to_predictions(self, logits):
        # CODE HERE
        pass
```

Improving the Model

Tips and advice on how to improve the BiLSTM model performance.

Chapter Goals:

- Learn how to improve the performance of the BiLSTM classifier

A. Model performance

The current iteration of the model will perform well on relatively simple datasets and can even perform adequately on larger datasets. However, there are still improvements that can be made to the model.

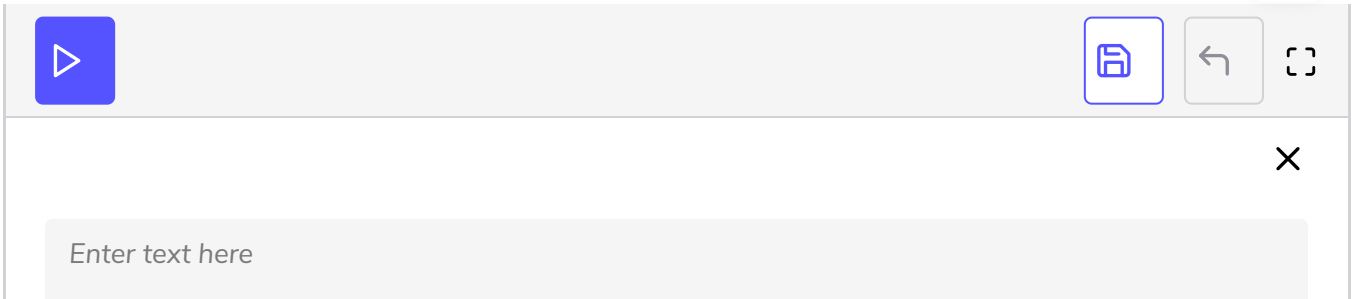
An easy way to improve the model would be to use more LSTM layers or hidden LSTM units. Our model only used a single LSTM layer with 512 hidden units, so adding more LSTM layers may benefit the model's performance. However, similar to adding hidden layers to a feed-forward neural network, it is important to make sure that additional LSTM layers don't lead to the model overfitting the training dataset.

B. Movie review sentiment analysis model

The code below runs a text classification model which has been implemented in the backend. The model classifies a given movie review as either positive or negative.

Paste in your own movie review to the text prompt and see whether the model thinks it’s positive or negative!

```
run_movie_model()
```



```
Press Run To Execute
```

Quiz

1

What is sentiment analysis?

COMPLETED 0%

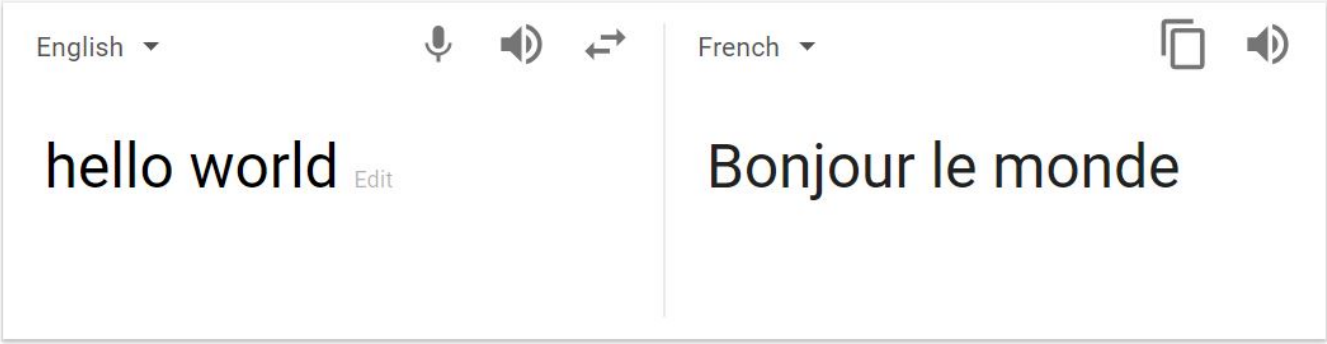
1 of 4

Introduction

In this section you will be building a sequence to sequence (seq2seq) model. Seq2Seq models are used for tasks that involve reading in a sequence of text and generating an output text sequence based on the input.

A. Sequence to sequence

One of the most important frameworks in NLP is the sequence to sequence (seq2seq) framework. This framework encompasses any task that involes taking in some text and returning some generated text. Dialog systems (e.g. chatbots), text summarization, and machine translation are all seq2seq applications.



A translation from English to French using Google Translate. This is an example of machine translation, a seq2seq task.

In the past, these types of seq2seq tasks were performed by regular statistical models, many of which utilized Bayesian statistics. However, with the rise of neural networks in the past decade, we've been able to apply deep learning to all these seq2seq tasks with great success.

In fact, there is an extremely powerful model called the *encoder-decoder*, which is specifically designed for seq2seq applications.

The encoder-decoder is named for its two parts: the encoder and the decoder. The encoder and the decoder are both language models (like the LSTM models

from previous sections).

First, an input sequence is fed to the encoder. The output from the last layer of the encoder becomes the input for the first layer of the decoder. The decoder transforms that input back into a text sequence.

In the following chapters you'll learn more about the encoder-decoder, as well as write code to build one yourself.

Training Data

Understand how training data is processed for sequence to sequence models.

Chapter Goals:

- Learn about the data used to train a seq2seq model
- Process input and output sequences into training data

A. Training task

For a seq2seq model, we use training pairs that contain an input sequence and an output sequence. For example, in machine translation the input sequence would be a sentence in one language, and the output sequence would be the sentence's correct translation in the other language.

During training, we perform two tasks:

1. **Input Task:** Extract useful information from the input sequence
2. **Output Task:** Calculate word probabilities at each output time step, using information from the input sequence and **previous words** in the output sequence

The input task is very common in NLP (e.g. text classification), and doesn't require any processing of the input sequence. The output task is equivalent to the task of language modeling, so it requires processing of the output sequence.

Specifically, we process the output sequence into two separate sequences: the ground truth sequence and the final token sequence.

B. Processing the output

The ground truth sequence for a seq2seq model is equivalent to the input sequence for a language model. It represents sequence prefixes that we use to calculate word probabilities at each time step of the output.

The final token sequence for a seq2seq model is equivalent to the output sequence when training a language model. It represents the "correct" words that the model should predict, based on the prefixes in the ground truth sequence.

Training Pair: ("he eats bread", "il mange du pain")

Ground Truth Sequence: ["il", "mange", "du"]

Final Token Sequence: ["mange", "du", "pain"]

Ground truth and final token sequences for an English-French machine translation model.

The ground truth and final token sequences are used to train a seq2seq model similar to how a language model is trained. The specific training details will be covered in later chapters when we introduce the encoder-decoder model.

C. SOS and EOS tokens

For seq2seq models, it is important to have start-of-sequence (SOS) and end-of-sequence (EOS) tokens. These tokens mark the start and end of a tokenized text sequence.

Input Sequence: ["SOS", "he", "eats", "bread", "EOS"]

Output Sequence: ["SOS", "il", "mange", "du", "pain", "EOS"]

Ground Truth Sequence: ["SOS", "il", "mange", "du"]

Final Token Sequence: ["mange", "du", "pain", "EOS"]

SOS and EOS tokens added to the training sequences from the previous example.

We will cover the reasoning behind using SOS and EOS tokens in later chapters. For now, just know that when the two tokens are added to the text corpus vocabulary, we refer to the new vocabulary as the *extended vocabulary*.

Time to Code!

In this section of the course you'll be creating a `Seq2SeqModel` object, which represents a seq2seq model.

Specifically, in this chapter you'll be completing the `make_training_tuple` function, which converts an input/output sequence pair into a training tuple.

The function is already filled with code that creates truncated versions of the output sequence and sets singleton lists for the SOS and EOS tokens. Your task is to use these truncated sequences and SOS/EOS tokens to create a tuple containing the update input, ground truth, and final token sequences.

For training the seq2seq model, the input sequence needs to have both the SOS and EOS tokens.

Set `input_sequence` equal to `input_sequence` with `sos_token` added to the front and `eos_token` added to the back.

The ground truth sequence is truncated in the back, so we only add the SOS token.

Set `ground_truth` equal to `truncate_back` with `sos_token` added to the front.

The final token sequence for the seq2seq model is truncated in the front, so we only add the EOS token.

Set `final_sequence` equal to `truncate_front` with `eos_token` added to the back.

Return a tuple containing `input_sequence` as the first element, `ground_truth` as the second, and `final_sequence` as the third.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    # Create a sequence training tuple from input/output sequences
    def make_training_tuple(self, input_sequence, output_sequence):
        truncate_front = output_sequence[1:]
        truncate_back = output_sequence[:-1]
        sos_token = [self.vocab_size]
        eos_token = [self.vocab_size + 1]
        # CODE HERE
```

Final States

Learn about the final state output of an LSTM and BiLSTM.

Chapter Goals:

- Learn about the final states for an LSTM and BiLSTM

A. The encoder

In an encoder-decoder model for seq2seq tasks, there are two components: the encoder and the decoder. The encoder is responsible for extracting useful information from the input sequence. For NLP applications, the encoder is normally an LSTM or BiLSTM.

B. LSTM final state

When using an LSTM or BiLSTM encoder, we need to pass the final state of the encoder into the decoder. The final state of an LSTM in TensorFlow is represented by the `LSTMStateTuple` object.

```
import tensorflow as tf

# Input sequences (embedded)
# Shape: (batch_size, max_seq_len, embed_dim)
input_embeddings = tf.placeholder(
    tf.float32, shape=(None, None, 4))

cell = tf.nn.rnn_cell.LSTMCell(5)
_, final_state = tf.nn.dynamic_rnn(
    cell,
    input_embeddings,
    dtype=tf.float32)

# final_state is the output of our LSTM encoder.
# it contains all the information about our input sequence,
# which in this case is just a tf.Placeholder object
print(final_state)
```

The final state of an LSTM, i.e. the second element of the returned tuple from tf.nn.dynamic_rnn.

An `LSTMStateTuple` object contains two important properties: the hidden state (`c`) and the state output (`h`). The hidden state represents the internal cell state (i.e. "memory") of the LSTM cell.

These two properties are represented by tensors with shape `(batch_size, hidden_units)`.

C. Multi-layer final states

For a multi-layer LSTM, the final state output of `dynamic_rnn` is a tuple containing the final state for each layer.

```
import tensorflow as tf

# Input sequences (embedded)
# Shape: (batch_size, max_seq_len, embed_dim)
input_embeddings = tf.placeholder(
    tf.float32, shape=(None, None, 4))

cell1 = tf.nn.rnn_cell.LSTMCell(5)
cell2 = tf.nn.rnn_cell.LSTMCell(8)
multi_cell = tf.nn.rnn_cell.MultiRNNCell([cell1, cell2])
_, final_state = tf.nn.dynamic_rnn(
    multi_cell,
    input_embeddings,
    dtype=tf.float32)

print(len(final_state))
print('\n')

print(final_state[0]) # layer 1
print('\n')

print(final_state[1]) # layer 2
print('\n')
```

The final state of a two-layer LSTM.

D. BiLSTM final state

The final state of a BiLSTM consists of the final state for the forward LSTM and the final state for the backward LSTM. This corresponds to a tuple containing two `LSTMStateTuple` objects.

```
import tensorflow as tf

# Input sequences (embedded)

# Shape: (batch_size, max_seq_len, embed_dim)
input_embeddings = tf.placeholder(
    tf.float32, shape=(None, None, 4))

cell_fw = tf.nn.rnn_cell.LSTMCell(5)
cell_bw = tf.nn.rnn_cell.LSTMCell(5)
_, final_states = tf.nn.bidirectional_dynamic_rnn(
    cell_fw,
    cell_bw,
    input_embeddings,
    dtype=tf.float32)

print(len(final_states))
print('\n')

print(final_states[0]) # forward
print('\n')

print(final_states[1]) # backward
print('\n')
```

The final state of a BiLSTM, i.e. the second element of the returned tuple from tf.nn.bidirectional_dynamic_rnn.

When the BiLSTM has multiple layers, the `final_states` output of `bidirectional_dynamic_rnn` is a tuple containing the final forward and backward states for each layer.

```
import tensorflow as tf

# Input sequences (embedded)
# Shape: (batch_size, max_seq_len, embed_dim)
input_embeddings = tf.placeholder(
    tf.float32, shape=(None, None, 4))

cell_fw1 = tf.nn.rnn_cell.LSTMCell(5)
cell_fw2 = tf.nn.rnn_cell.LSTMCell(9)
multi_cell_fw = tf.nn.rnn_cell.MultiRNNCell(
    [cell_fw1, cell_fw2])

cell_bw1 = tf.nn.rnn_cell.LSTMCell(5)
cell_bw2 = tf.nn.rnn_cell.LSTMCell(9)
multi_cell_bw = tf.nn.rnn_cell.MultiRNNCell(
    [cell_bw1, cell_bw2])

_, final_states = tf.nn.bidirectional_dynamic_rnn(
    multi_cell_fw,
    multi_cell_bw,
    input_embeddings,
    dtype=tf.float32)
```

```
final_fw, final_bw = final_states


print(final_fw[0]) # forward layer 1
print('\n')

print(final_fw[1]) # forward layer 2
print('\n')

print(final_bw[0]) # backward layer 1
print('\n')

print(final_bw[1]) # backward layer 2
print('\n')
```

The final forward and backward state tuples for a two-layer BiLSTM.

E. Combining forward and backward

In order to use BiLSTM final states in an encoder-decoder model, we need to combine the forward and backward states. This is because the decoder portion utilizes a regular LSTM, which only has a forward direction.

Luckily, combining the forward and backward states is rather simple. The main thing we need to do is concatenate the hidden state and state output of both the forward and backward states.

```
import tensorflow as tf

# Forward state of single-layer BiLSTM final states
fw_c, fw_h = final_states[0].c, final_states[0].h

# Backward state of single-layer BiLSTM final states
bw_c, bw_h = final_states[1].c, final_states[1].h

# Concatenate along final axis
final_c = tf.concat([fw_c, bw_c], -1)
final_h = tf.concat([fw_h, bw_h], -1)
```

Concatenating the hidden state (c) and state output (h). Note that the concatenation is applied along the final axis.

We'll describe in the next chapter how to convert the combined values into an `LSTMStateTuple` object.

Time to Code!

In this chapter you'll be completing the `get_bi_state_parts` function, which combines the forward and backward final states for a layer in the BiLSTM. The function is used as a helper in the `Seq2Seq` object's `encoder` function, which creates the encoder portion of the model.

When combining the forward and backward states, we use the `tf.concat` function to concatenate along the final axis. We perform separate concatenations for the hidden state (`c`) and the state output (`h`).

Set `bi_state_c` equal to `tf.concat` applied with a list containing `state_fw.c` and `state_bw.c` (in that order), and `-1` as the axis.

Set `bi_state_h` equal to `tf.concat` applied with a list containing `state_fw.h` and `state_bw.h` (in that order), and `-1` as the axis.

Return a tuple containing `bi_state_c` as the first element and `bi_state_h` as the second.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Get c and h vectors for bidirectional LSTM final states
def get_bi_state_parts(state_fw, state_bw):
    # CODE HERE
    pass

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    # Create multi-layer LSTM
    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell
```

```
    # Get embeddings for input/output sequences
    def get_embeddings(self, sequences, scope_name):
        with tf.variable_scope(scope_name):

            cat_column = tf_fc.sequence_categorical_column_with_identity(
                'sequences',
                self.extended_vocab_size)
            embedding_column = tf.feature_column.embedding_column(
                cat_column,
                int(self.extended_vocab_size**0.25))
            seq_dict = {'sequences': sequences}
            embeddings, sequence_lengths = tf_fc.sequence_input_layer(
                seq_dict,
                [embedding_column])
            return embeddings, tf.cast(sequence_lengths, tf.int32)

    # Create the encoder for the model
    def encoder(self, encoder_inputs, is_training):
        input_embeddings, input_seq_lens = self.get_embeddings(encoder_inputs, 'encoder_emb')
        cell_fw = self.stacked_lstm_cells(is_training, self.num_lstm_units)
        cell_bw = self.stacked_lstm_cells(is_training, self.num_lstm_units)
        enc_outputs, final_states = tf.nn.bidirectional_dynamic_rnn(
            cell_fw,
            cell_bw,
            input_embeddings,
            sequence_length=input_seq_lens,
            dtype=tf.float32)
        states_fw, states_bw = final_states
        combined_state = []
        for i in range(self.num_lstm_layers):
            bi_state_c, bi_state_h = get_bi_state_parts(
                states_fw[i], states_bw[i]
            )
```

Combined State

Combine the final states for a BiLSTM into usable initial states.

Chapter Goals:

- Combine the final states for each BiLSTM layer

A. LSTMStateTuple initialization

We initialize an `LSTMStateTuple` object with a hidden state (`c`) and state output (`h`).

Below we show an example of initializing an `LSTMStateTuple` object using the final forward and backward states from a single layer BiLSTM encoder.

```
import tensorflow as tf

# Final states of single-layer BiLSTM
# Forward and backward cells both have 5 hidden units
state_fw, state_bw = final_states

# Concatenate along final axis
final_c = tf.concat([state_fw.c, state_bw.c], -1)
final_h = tf.concat([state_fw.h, state_bw.h], -1)

combined_state = tf.nn.rnn_cell.LSTMStateTuple(
    final_c, final_h)
print(combined_state)
```

In the above example, we combined the BiLSTM forward and backward states into a single `LSTMStateTuple` object, which can be passed into the decoder.

For BiLSTM encoders with multiple layers, we combine the states for each layer to create a tuple of `LSTMStateTuple` objects. The element at index i of the tuple is the ith layer's combined final state.

Time to Code!

In this chapter you'll be finishing the `for` loop of the `encoder` function. This is on line **65** of the code editor.

For each BiLSTM layer, we create a combined state using the combined `c` and `h` properties from the previous chapter.

Inside the `for` loop, set `bi_lstm_state` equal to `tf.nn.rnn_cell.LSTMStateTuple`, initialized with `bi_state_c` and `bi_state_h`.

After creating the layer's final state, we append it to the end of `combined_state`.

Inside the `for` loop, append `bi_lstm_state` to the end of `combined_state`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Get c and h vectors for bidirectional LSTM final states
def get_bi_state_parts(state_fw, state_bw):
    bi_state_c = tf.concat([state_fw.c, state_bw.c], -1)
    bi_state_h = tf.concat([state_fw.h, state_bw.h], -1)
    return bi_state_c, bi_state_h

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    # Create multi-layer LSTM
    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell

    # Get embeddings for input/output sequences
    def get_embeddings(self, sequences, scope_name):
        with tf.variable_scope(scope_name):
            cat_column = tf_fc.sequence_categorical_column_with_identity(
                'sequences',
                self.extended_vocab_size)
```

```
        self.extended_vocab_size)
        embedding_column = tf.feature_column.embedding_column(
            cat_column,
            int(self.extended_vocab_size**0.25))
        seq_dict = {'sequences': sequences}
        embeddings, sequence_lengths = tf_fc.sequence_input_layer(
            seq_dict,
            [embedding_column])
        return embeddings, tf.cast(sequence_lengths, tf.int32)

    # Create the encoder for the model
    def encoder(self, encoder_inputs, is_training):
        input_embeddings, input_seq_lens = self.get_embeddings(encoder_inputs, 'encoder_emb')
        cell_fw = self.stacked_lstm_cells(is_training, self.num_lstm_units)
        cell_bw = self.stacked_lstm_cells(is_training, self.num_lstm_units)
        enc_outputs, final_states = tf.nn.bidirectional_dynamic_rnn(
            cell_fw,
            cell_bw,
            input_embeddings,
            sequence_length=input_seq_lens,
            dtype=tf.float32)
        states_fw, states_bw = final_states
        combined_state = []
        for i in range(self.num_lstm_layers):
            bi_state_c, bi_state_h = get_bi_state_parts(
                states_fw[i], states_bw[i]
            )
            # CODE HERE
        final_state = tuple(combined_state)
        return enc_outputs, input_seq_lens, final_state
```

Encoder-Decoder

Learn about the encoder-decoder model architecture.

Chapter Goals:

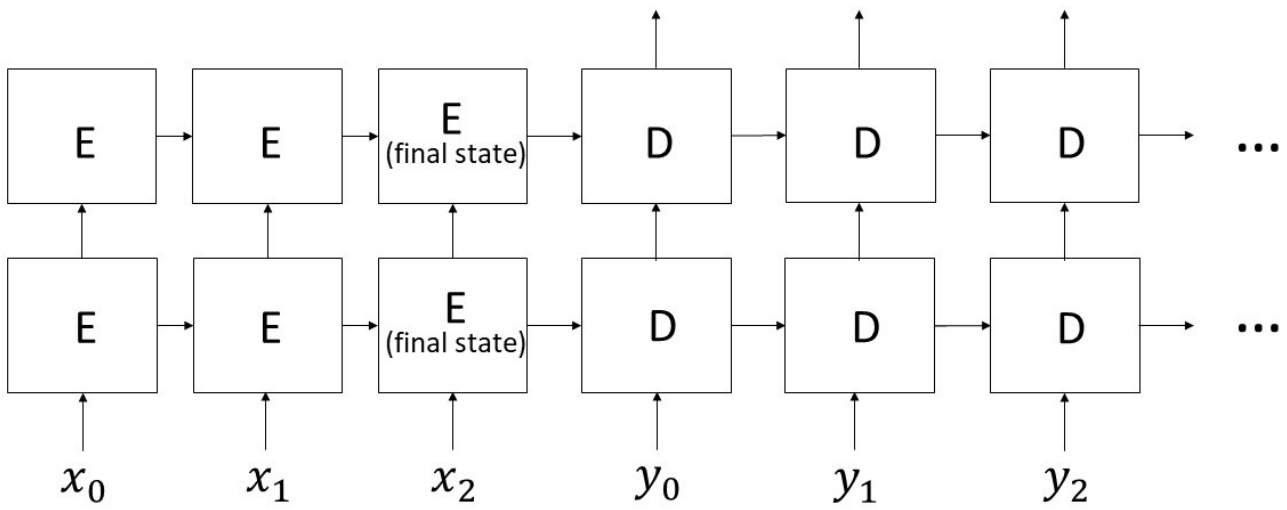
- Learn about the encoder-decoder model architecture

A. Model architecture

As previously mentioned, the encoder portion of the encoder-decoder model in this section is a BiLSTM. The decoder portion is just a regular forward LSTM, with the same number of LSTM layers as the encoder.

What makes an encoder-decoder model so powerful is that the decoder uses the final state of the encoder as its initial state. This gives the decoder access to the information that the encoder extracted from the input sequence, which is crucial for good sequence to sequence modeling.

In the case of multiple layers for the encoder-decoder model, each layer's final state in the encoder acts as the initial state for the corresponding layer in the decoder.



An unrolled encoder-decoder model with two LSTM layers. The encoder cells at each time step are labeled with "E", while the decoder's are labeled with "D".

In the above diagram, the final state of the LSTM encoder for each layer is used as the starting state for the corresponding decoder layer. Note that the encoder does not return outputs. This is because, in an encoder-decoder model, we ignore the encoder's outputs and only use the outputs of the decoder.

The input tokens for the encoder represent the input sequence for the encoder-decoder model. The input tokens for the decoder represent the ground truth tokens for the given input sequence. The decoder's output is equivalent to the output of a language model, i.e. each time step's output is based on the ground truth tokens from the previous time steps.

Note that the encoder in this diagram is a regular LSTM, rather than a BiLSTM. In the case of a BiLSTM, the starting state for each decoder layer is the combined final state from the corresponding BiLSTM layer (which you computed in the previous chapter).

B. Training vs. inference

During training, we have access to both the input and output sequences for each training pair. This gives the decoder access to the ground truth tokens.

However, during inference (i.e. real-time predictions), the model only has access to the input sequence. This means that the decoder doesn't have access to ground truth tokens. We handle this by using the *decoder's* word predictions from previous time steps to replace the ground truth tokens.

This will be covered in greater detail in chapter 11, when we discuss methods for decoding during inference.

Time to Code!

In this chapter you'll be writing some code in the `create_decoder_cell` function (which you'll subsequently complete in the next chapter). The function creates stacked LSTM cells for the decoder.

Each of the decoder LSTM cells contains twice the number of hidden units as the encoder LSTM cells to match the concatenated size of the encoder's forward and backward final states. We provide a function called

`stacked_lstm_cells` which creates a (multi-layered) LSTM cell.

The `stacked_lstm_cells` function takes in the `is_training` boolean as the first argument and the number of hidden units as the second argument.

Set `num_decode_units` equal to two times `self.num_lstm_units`.

Set `dec_cell` equal to `self.stacked_lstm_cells` with `is_training` and `num_decode_units` as the arguments.

In the **Text Classification** section, we combined the forward and backward outputs of a BiLSTM in order to use them. We provide a utility function called `combine_enc_outputs` which does this for you.

Set `combined_enc_outputs` equal to `self.combine_enc_outputs` applied to `enc_outputs`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    # Create multi-layer LSTM cells
    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell

    # Helper funtion to combine BiLSTM encoder outputs
    def combine_enc_outputs(self, enc_outputs):
        enc_outputs_fw, enc_outputs_bw = enc_outputs
        return tf.concat([enc_outputs_fw, enc_outputs_bw], -1)

    # Create the stacked LSTM cells for the decoder
    def create_decoder_cell(self, enc_outputs, input_seq_lens, is_training):
        # CODE HERE
        pass
```

Attention

Learn about the attention mechanism and why it's important.

Chapter Goals:

- Learn about attention and understand why it's useful
- Incorporate attention into the decoder LSTM

A. Using the encoder

Based on the encoder-decoder model architecture, the only thing that the decoder gets from the encoder is the final state in each layer. The final states basically encapsulate the encoder's extracted information from the input sequence, which is passed into the decoder.

However, trying to encapsulate all the useful information from an input sequence into a final state is a difficult task, especially if the input sequence is large and contains long-term dependencies. This is a problem that has been shown to exist in practice, where decoders perform poorly on input sequences with long-term dependencies.

> **"Sam grew up in Los Angeles. As a child, he dreamed of one day becoming an actor like Brad Pitt or Johnny Depp. Each day he would practice public speaking and impromptu skits near Venice Beach."**

An input text sequence with many long-term dependencies. A regular encoder-decoder model would struggle to decode this input, e.g. for a task like translation.

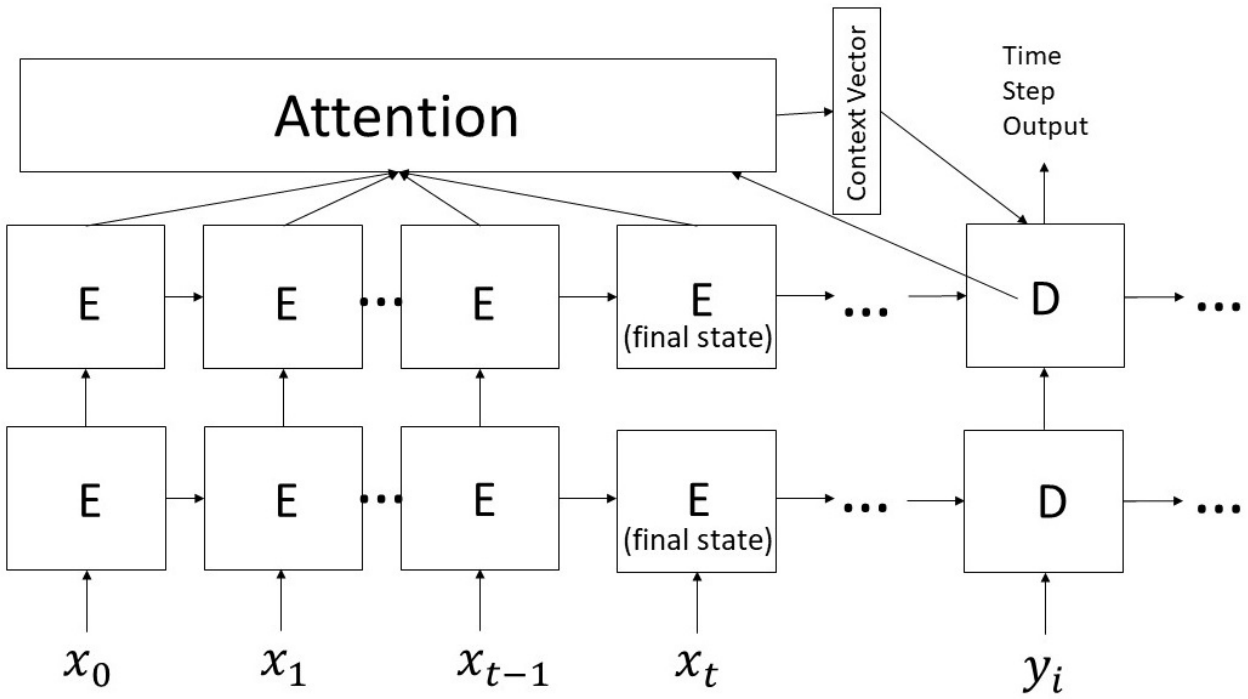
The obvious solution to this issue is to give the decoder access to each of the encoder's intermediate time step outputs. In the previous chapter's diagram, the encoder's outputs were not used. However, if we were to use the encoder's outputs as additional input for the decoder, it would give the decoder a lot more useful information about the input sequence. The way we do this is by using *attention*.

B. How attention works

B. How attention works

Although we want to use the input sequence for the decoder, we don't necessarily want to use each input token equally. With attention, we let the decoder decide which encoder outputs are most useful for the decoder at the current decoding time step.

Using the decoder's hidden state at the current time step, as well as the encoder outputs, attention will calculate something called a *context vector*. The context vector encapsulates the most meaningful information from the input sequence for the current decoder time step, and it's used as additional input for the decoder when calculating the time step's output.



Using attention to calculate a context vector for the current decoder time step (time step i). The context vector is used to obtain the decoder output at time step i.

Attention makes use of trainable weights to calculate a context vector. You can think of attention as a mini neural network, which takes as input the decoder's current state and the encoder outputs, and uses its trainable weights to produce a context vector.

C. Attention mechanisms

The exact process for computing the context vector depends on the *attention mechanism* that's used. There are quite a few attention mechanism variations, but the two most popular ones in TensorFlow are `BahdanauAttention` and

`LuongAttention`. These two attention mechanisms are named after their main inventors, Dzmitry Bahdanau and Minh-Thang Luong, respectively.

The main difference between the two mechanisms is how they combine the encoder outputs and current time step hidden state when computing the context vector. The Bahdanau mechanism uses an *additive* (concatenation-based) method, while the Luong mechanism uses a multiplicative method.

In our code we use the Luong mechanism because it has been shown to work better in certain seq2seq tasks, such as machine translation. However, both TensorFlow mechanisms are initialized in nearly identical fashion.

Below we show how to initialize the two attention mechanisms as TensorFlow objects.

```
import tensorflow as tf

# Placeholder representing the
# individual lengths of each input sequence in the batch
input_seq_lens = tf.placeholder(tf.int32, shape=(None,))

num_units = 8
bahdanau = tf.contrib.seq2seq.BahdanauAttention(
    num_units,
    # combined encoder outputs (from previous chapter)
    combined_enc_outputs,
    memory_sequence_length=input_seq_lens)
luong = tf.contrib.seq2seq.LuongAttention(
    num_units,
    # combined encoder outputs (from previous chapter)
    combined_enc_outputs,
    memory_sequence_length=input_seq_lens)
```

The first required argument for the attention mechanisms is just the number of hidden units in the encoder outputs, i.e. the final dimension size. The second required argument is the encoder outputs. The `memory_sequence_length` keyword argument is a tensor representing the length of each input sequence, and it's used to ensure that attention is only applied at non-padding time steps.

D. TensorFlow AttentionWrapper

While the purpose and high-level usage of attention is easy enough to understand, the implementation requires quite a bit of linear algebra and

advanced mathematics. Luckily, TensorFlow gives us an easy-to-use API for adding attention to an LSTM decoder cell via the `AttentionWrapper` function. Below is an example of how to use the `AttentionWrapper` function to add attention to the decoder cell.

```
import tensorflow as tf

# Decoder LSTM cell
dec_cell = tf.nn.rnn_cell.LSTMCell(8)
dec_cell = tf.contrib.seq2seq.AttentionWrapper(
    dec_cell,
    luong, # LuongAttention object
    attention_layer_size=8)
```

The required arguments for the `AttentionWrapper` function are the decoder cell and the attention mechanism to use. Of the keyword arguments, the one we use in the code is `attention_layer_size`.

When using the decoder in TensorFlow, we pass along the *attention value* at each decoder time step into the cell state at the next time step. The default behavior is to just pass along the context vector as the attention value. However, if we set the `attention_layer_size` or `attention_layer` keyword arguments, a fully-connected layer will combine the context vector with the time step's output. The output of the fully-connected layer is used as the attention value.

Using a fully-connected layer to create the attention value can benefit the model's performance, by utilizing the decoder's outputs as additional information. The value used for `attention_layer_size` specifies the number of hidden units in the fully-connected layer. In the example above, we set it equal to the number of decoder hidden units.

Time to Code!

In this chapter you'll be completing the `create_decoder_cell` function to use attention.

The attention mechanism we'll use for the decoder is `LuongAttention`. Note that we provide a shorthand variable for the `tf.contrib.seq2seq` module, called `tf.c2s`.

called `tf_s2s`.

Set `attention_mechanism` equal to `tf_s2s.LuongAttention` applied with `num_decode_units` and `combined_enc_outputs` as the required arguments, along with `input_seq_lens` as the `memory_sequence_length` keyword argument.

Using the attention mechanism, we can apply an `AttentionWrapper` around the `dec_cell` variable initialized in the previous chapter.

Set `dec_cell` equal to `tf_s2s.AttentionWrapper` applied with `dec_cell` and `attention_mechanism` as the required arguments, along with `num_decode_units` as the `attention_layer_size` keyword argument.

After wrapping the decoder cell with the attention mechanism, we return the cell.

Return `dec_cell`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    # Create multi-layer LSTM cells
    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell

    # Helper funtion to combine BiLSTM encoder outputs
    def combine_enc_outputs(self, enc_outputs):
        enc_outputs_fw, enc_outputs_bw = enc_outputs
        return tf.concat([enc_outputs_fw, enc_outputs_bw], -1)

    # Create the stacked LSTM cells for the decoder
    def create_decoder_cell(self, enc_outputs, input_seq_lens, is_training):
```

```
    def create_decoder_cell(self, enc_outputs, input_seq_lens, is_training):
        num_decode_units = self.num_lstm_units * 2
        dec_cell = self.stacked_lstm_cells(is_training, num_decode_units)

        combined_enc_outputs = self.combine_enc_outputs(enc_outputs)
        # CODE HERE
```

Training Helper

Create a Training Helper object for training the decoder.

Chapter Goals:

- Learn about the decoding process during training
- Create a `TrainingHelper` object to use for decoding

A. Decoding during training

During training, we have access to both the input and output sequences of a training pair. This means that we can use the output sequence's ground truth tokens as input for the decoder.

We'll get into the specifics of TensorFlow decoding, i.e. using the decoder to generate model outputs, in later chapters. However, before we perform any decoding, we need to set up a `Helper` object. For training, the `Helper` object instance we use is the `TrainingHelper`.

Below, we create a `TrainingHelper` object for decoding during training.

```
import tensorflow as tf

# Placeholder representing the
# batch of (embedded) input sequences for the decoder
# Shape: (batch_size, max_seq_len, embed_dim)
decoder_embeddings = tf.placeholder(
    tf.float32, shape=(None, None, 12)
)

# Placeholder representing the
# individual lengths of each input sequence in the batch
decoder_seq_lens = tf.placeholder(tf.int32, shape=(None,))

helper = tf.contrib.seq2seq.TrainingHelper(
    decoder_embeddings, decoder_seq_lens)
```

The `TrainingHelper` object is initialized with the (embedded) ground truth

The `TrainingHelper` object is initialized with the (embedded) ground truth sequences and the lengths of the ground truth sequences. Note that we use

separate embedding models for the encoder input and the decoder input (i.e. ground truth tokens). This is because there are different word relationships in the input and output sequences for a seq2seq task, and sometimes the sequences can be completely different (e.g. machine translation).

Time to Code!

In this chapter you'll be filling in part of the `create_decoder_helper` function, which creates the Helper object for the decoder. Specifically, in this chapter you'll be focusing on creating the `TrainingHelper` object.

We use the decoder differently depending on whether we're in training or inference mode. If we're in training mode, we use the `decoder_inputs` as the ground truth tokens.

Before using the ground truth tokens, we need to convert them into embeddings. We provide a function called `get_embeddings`, which returns the embeddings and sequence lengths. The function takes in the input sequences and the name of the embedding model as required arguments.

Inside the `if` block, set the tuple `dec_embeddings, dec_seq_lens` equal to `self.get_embeddings` applied with `decoder_inputs` and `'decoder_emb'` as the required arguments.

During training, we use the `TrainingHelper` object. It takes in the ground truth embeddings and sequence lengths as required arguments.

Inside the `if` block, set `helper` equal to `tf_s2s.TrainingHelper` applied with `dec_embeddings` and `dec_seq_lens` as the required arguments.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
```

```
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    # Convert sequences to embeddings
    def get_embeddings(self, sequences, scope_name):
        with tf.variable_scope(scope_name):
            cat_column = tf_fc.sequence_categorical_column_with_identity(
                'sequences',
                self.extended_vocab_size)
            embedding_column = tf.feature_column.embedding_column(
                cat_column,
                int(self.extended_vocab_size**0.25))
            seq_dict = {'sequences': sequences}
            embeddings, sequence_lengths = tf_fc.sequence_input_layer(
                seq_dict,
                [embedding_column])
            return embeddings, tf.cast(sequence_lengths, tf.int32)

    # Create the helper for decoding
    def create_decoder_helper(self, decoder_inputs, is_training, batch_size):
        if is_training:
            # CODE HERE
            pass
        else:
            # IGNORE FOR NOW
            pass
        return helper, dec_seq_lens
```

Decoder Object

Learn about the decoder object for the encoder-decoder model.

Chapter Goals:

- Convert the encoder's final state into the proper format for decoding with attention
- Create a `BasicDecoder` object to use for decoding

A. Creating the initial state

The final state from the encoder is a tuple containing an `LSTMStateTuple` object for each layer of the BiLSTM. However, if we want to use this as the initial state for an attention-wrapped decoder, we need to convert it into an `AttentionWrapperState`.

The conversion is a two step process. We first use the `zero_state` function of the attention-wrapped decoder cell to create a blank `AttentionWrapperState`. We then use the `clone` function to copy the encoder's final state into the blank `AttentionWrapperState`. The result is the initial state for the attention-wrapped decoder.

Below we demonstrate how to create the decoder's initial state from the encoder's final state.

```
import tensorflow as tf

batch_size = 10
# dec_cell is an attention-wrapped decoder LSTM cell
zero_cell = dec_cell.zero_state(batch_size, tf.float32)

# final_state is the final encoder state of a 2-layer BiLSTM
initial_state = zero_cell.clone(cell_state=final_state)
```

The `zero_state` function takes the batch size and the state's type (normally

`tf.float32`) as required arguments. We set the `clone` function's `cell_state` keyword argument with the encoder's final state to create the decoder's initial state.

B. The BasicDecoder object

The decoder object that we use for decoding is the `BasicDecoder`. To create an instance of the `BasicDecoder` object, we need to pass in the decoder cell, helper object, and initial decoder state as required arguments.

```
import tensorflow as tf

decoder = tf.contrib.seq2seq.BasicDecoder(
    dec_cell, helper, initial_state)
```

Creating a BasicDecoder object.

The `BasicDecoder` constructor has a keyword argument called `output_layer`, which can be used to apply a fully-connected layer to the model's outputs. This is a nice shortcut when calculating the model's logits.

```
import tensorflow as tf

num_units = 500 # extended vocab size
projection_layer = tf.layers.Dense(num_units)
decoder = tf.contrib.seq2seq.BasicDecoder(
    dec_cell, helper, initial_state,
    output_layer=projection_layer)
```

Creating a BasicDecoder object with direct logit calculation.

Instead of using the `tf.layers.dense` function, we create a fully-connected layer object using the `tf.layers.Dense` constructor. The required argument for initialization is the number of hidden units. In the example above, we set it equal to the extended vocabulary size, which gives us the proper shape for the logits.

Time to Code!

In this chapter you'll be completing the `create_basic_decoder` function, which is used in the model's `decoder` function to create a `BasicDecoder` object from a decoder LSTM cell and Helper object.

When creating the decoder object, we'll apply a projection layer to the end, which will calculate the model's logits directly.

Set `projection_layer` equal to `tf.layers.Dense` initialized with `extended_vocab_size`.

Before we can create the decoder object, we need to make sure that the initial state for the decoder is in the correct format.

We'll use the `zero_state` function to create a blank `AttentionWrapperState`, then use the `clone` function to copy the `final_state` contents into the blank `AttentionWrapperState`.

Set `zero_cell` equal to `dec_cell.zero_state` applied with `batch_size` and `tf.float32` as the required arguments.

Set `initial_state` equal to `zero_cell.clone` applied with `batch_size` as the `cell_state` keyword argument.

We can now create the decoder object using the `BasicDecoder` constructor. We'll use `dec_cell`, `helper`, and `initial_state` as the required arguments for initialization.

Set `decoder` equal to `tf_s2s.BasicDecoder` initialized with the specified required arguments as well as `projection_layer` for the `output_layer` keyword argument. Then return `decoder`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

def create_basic_decoder(extended_vocab_size, batch_size, final_state, dec_cell, helper):
    # CODE HERE
    pass

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
```

```
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers

        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell

    # Helper funtion to combine BiLSTM encoder outputs
    def combine_enc_outputs(self, enc_outputs):
        enc_outputs_fw, enc_outputs_bw = enc_outputs
        return tf.concat([enc_outputs_fw, enc_outputs_bw], -1)

    # Create the stacked LSTM cells for the decoder
    def create_decoder_cell(self, enc_outputs, input_seq_lens, is_training):
        num_decode_units = self.num_lstm_units * 2
        dec_cell = self.stacked_lstm_cells(is_training, num_decode_units)
        combined_enc_outputs = self.combine_enc_outputs(enc_outputs)
        attention_mechanism = tf_s2s.LuongAttention(
            num_decode_units, combined_enc_outputs,
            memory_sequence_length=input_seq_lens)
        dec_cell = tf_s2s.AttentionWrapper(
            dec_cell, attention_mechanism,
            attention_layer_size=num_decode_units)
        return dec_cell

    # Create the helper for decoding
    def create_decoder_helper(self, decoder_inputs, is_training, batch_size):
        if is_training:
            dec_embeddings, dec_seq_lens = self.get_embeddings(decoder_inputs, 'decoder_emb')
            helper = tf_s2s.TrainingHelper(
                dec_embeddings, dec_seq_lens)
        else:
            pass
        return helper, dec_seq_lens

    # Create the decoder for the model
    def decoder(self, enc_outputs, input_seq_lens, final_state, batch_size,
        decoder_inputs=None, maximum_iterations=None):
        is_training = decoder_inputs is not None
        dec_cell = self.create_decoder_cell(enc_outputs, input_seq_lens, is_training)
        helper, dec_seq_lens = self.create_decoder_helper(decoder_inputs, is_training, batch_
        decoder = create_basic_decoder(
            self.extended_vocab_size, batch_size, final_state, dec_cell, helper)
```

Decoding Output

Decode the model's outputs for training and inference.

Chapter Goals:

- Retrieve the decoder outputs and return the model's logits during training

After creating the decoder object for our model, we can perform the decoding using the `dynamic_decode` function.

```
import tensorflow as tf

extended_vocab_size = 500
batch_size = 10
# decoder is a BasicDecoder object
outputs = tf.contrib.seq2seq.dynamic_decode(decoder)

decoder_output = outputs[0]
logits = decoder_output.rnn_output
decoder_final_state = outputs[1]
decoded_sequence_lengths = outputs[2]
```

Using the dynamic_decode function for decoding and producing logits.

The `dynamic_decode` function takes in one required argument, which is the decoder object. It returns a tuple containing three elements:

1. The decoder's output. For a `BasicDecoder` input, the decoder's output takes the form of a `BasicDecoderOutput` object.
2. The decoder's final state. This isn't used in our encoder-decoder model.
3. The lengths of each of the decoder's output sequences. This also isn't used in our encoder-decoder model.

If the `BasicDecoder` input was initialized with the `output_layer` keyword argument, the `rnn_output` of the `BasicDecoderOutput` object will be the model's

logits

B. Limiting the decoded length

A problem that sometimes occurs when decoding, especially in tasks like text summarization, is the decoder returning output sequences that are too long. We can manually limit the decoder output length with the `dynamic_decode` function's `maximum_iterations` keyword argument.

By setting this keyword argument with an integer, k, we guarantee that the decoder will not output sequences with length longer than k.

Time to Code!

In this chapter you'll be completing the `run_decoder` function, which is used in the model's `decoder` function to run the decoder object.

We can run the decoder object using the `dynamic_decode` function. We only care about the first element in the returned tuple, which contains the output of our decoder.

Set `dec_outputs` equal to the first element of the tuple returned by `tf_s2s.dynamic_decode`. Call the function with `decoder` as the required argument and `maximum_iterations` for the `maximum_iterations` keyword argument.

During training, the model's logits are located in the `rnn_output` property of `dec_outputs`. The `decoder` function will return both the model's logits and the ground truth sequence lengths.

Create an `if` block that checks if `is_training` is `True`. Inside the `if` block, return a tuple containing the model's logits and `dec_seq_lens`, in that order.

When we're not training, we will return the model's predictions, which is in the `sample_id` attribute of `dec_outputs` (more on this later).

Outside the `if` block, return `dec_outputs.sample_id`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq
```

```
def run_decoder(decoder, maximum_iterations, dec_seq_lens, is_training):
    # CODE HERE
    pass

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    def make_lstm_cell(self, dropout_keep_prob, num_units):
        cell = tf.nn.rnn_cell.LSTMCell(num_units)
        return tf.nn.rnn_cell.DropoutWrapper(cell, output_keep_prob=dropout_keep_prob)

    def stacked_lstm_cells(self, is_training, num_units):
        dropout_keep_prob = 0.5 if is_training else 1.0
        cell_list = [self.make_lstm_cell(dropout_keep_prob, num_units) for i in range(self.nu
        cell = tf.nn.rnn_cell.MultiRNNCell(cell_list)
        return cell

    # Helper funtion to combine BiLSTM encoder outputs
    def combine_enc_outputs(self, enc_outputs):
        enc_outputs_fw, enc_outputs_bw = enc_outputs
        return tf.concat([enc_outputs_fw, enc_outputs_bw], -1)

    # Create the stacked LSTM cells for the decoder
    def create_decoder_cell(self, enc_outputs, input_seq_lens, is_training):
        num_decode_units = self.num_lstm_units * 2
        dec_cell = self.stacked_lstm_cells(is_training, num_decode_units)
        combined_enc_outputs = self.combine_enc_outputs(enc_outputs)
        attention_mechanism = tf_s2s.LuongAttention(
            num_decode_units, combined_enc_outputs,
            memory_sequence_length=input_seq_lens)
        dec_cell = tf_s2s.AttentionWrapper(
            dec_cell, attention_mechanism,
            attention_layer_size=num_decode_units)
        return dec_cell

    # Create the helper for decoding
    def create_decoder_helper(self, decoder_inputs, is_training, batch_size):
        if is_training:
            dec_embeddings, dec_seq_lens = self.get_embeddings(decoder_inputs, 'decoder_emb')
            helper = tf_s2s.TrainingHelper(
                dec_embeddings, dec_seq_lens)
        else:
            pass
        return helper, dec_seq_lens

    # Create the decoder for the model
    def decoder(self, enc_outputs, input_seq_lens, final_state, batch_size,
        decoder_inputs=None, maximum_iterations=None):
        is_training = decoder_inputs is not None
        dec_cell = self.create_decoder_cell(enc_outputs, input_seq_lens, is_training)
        helper, dec_seq_lens = self.create_decoder_helper(decoder_inputs, is_training, batch_
        projection_layer = tf.layers.Dense(self.extended_vocab_size)
        zero_cell = dec_cell.zero_state(batch_size, tf.float32)
        initial_state = zero_cell.clone(cell_state=final_state)
```

```
        decoder = tf_s2s.BasicDecoder(
            dec_cell, helper, initial_state,

            output_layer=projection_layer)
        return run_decoder(decoder, maximum_iterations, dec_seq_lens, is_training)
```

Calculating Loss

Calculate the model's loss based on logits and sparse outputs.

Chapter Goals:

- Calculate the training loss based on the model's logits and final token sequences

A. Final token sequence

So far, we've used the input sequences and ground truth sequences for training the encoder-decoder model. The final token sequences are used when calculating the loss.

If we view the decoder as a language model, the ground truth sequences act as the language model's input while the final token sequences act as the "correct" output for the language model.

In a language model, we calculate the loss based on the model's logits and the "correct" output. For the encoder-decoder, the loss is calculated in the same way. We use a sparse softmax cross entropy loss based on the logits and final token sequences, and then zero-out the time steps that correspond to padding.

B. Sequence mask

Previously, we used the `tf.sign` function to create binary sequences that are used to zero-out padding time steps. However, there is another way to create the binary sequences, also known as a *sequence mask*, using just the sequence lengths of the decoder's inputs.

The function that provides this utility is called `tf.sequence_mask`. It takes in a tensor of sequence lengths as its required argument, and returns the corresponding sequence mask.

```
import tensorflow as tf

# Example sequence lengths
```

```
# Example sequence lengths
seq_lens = tf.constant([3, 4, 2])
binary_sequences = tf.sequence_mask(seq_lens)
int_sequences = tf.sequence_mask(seq_lens, dtype=tf.int32)

with tf.Session() as sess:
    binary_array = sess.run(binary_sequences)
    int_array = sess.run(int_sequences)

print(repr(binary_array))
print(repr(int_array))
```

Creating sequence masks from sequence lengths using tf.sequence_mask.

Note that the default return type of `tf.sequence_mask` is a boolean tensor. We can set the `dtype` keyword argument to specify a different return type.

Time to Code!

In this chapter you'll be completing the `calculate_loss` function, which is used to calculate the model's loss.

When calculating the loss, we need to zero-out any time steps that represent padding, i.e. time steps past the actual sequence length. We'll apply `tf.sequence_mask` to the decoder's input sequence lengths (`dec_seq_lens`) to obtain the necessary sequence mask.

Set `binary_sequences` equal to `tf.sequence_mask` applied with `dec_seq_lens` as the required argument. Also set the `dtype` keyword argument to `tf.float32`, so that `binary_sequences` has type `tf.float32`.

Just like in language modeling, we use sparse softmax cross entropy as the loss function. We also zero-out the loss at time steps that represent padding.

Set `batch_loss` equal to `tf.nn.sparse_softmax_cross_entropy_with_logits` applied with `decoder_outputs` and `logits` as the `labels` and `logits` keyword arguments, respectively.

Set `unpadded_loss` equal to `batch_loss` multiplied by `binary_sequences`.

The loss that we return will be the per-sequence loss, the average loss for each sequence in the training batch. This is equivalent to taking the entire loss and dividing it by the batch size.

Set `per_seq_loss` equal to `tf.reduce_sum` applied to `unpadded_loss`, divided by `batch_size`. Then return `per_seq_loss`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    # Calculate the model loss
    def calculate_loss(self, logits, dec_seq_lens, decoder_outputs, batch_size):
        # CODE HERE
        pass
```

Inference Decoding

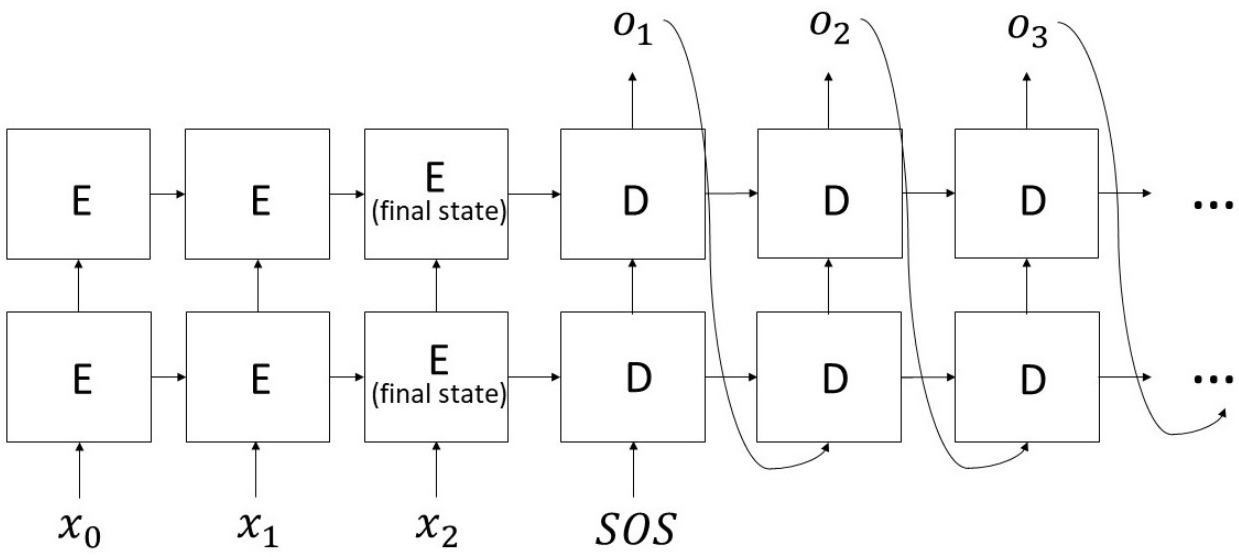
Understand the difference between training and inference decoding.

Chapter Goals:

- Learn how to perform decoding for inference
- Learn about variable scopes for declaring and retrieving variables

A. Decoding without ground truth

During training, we have access to ground truth tokens, which are used as input for the decoder. However, for inference (i.e. generating output in real-time), we only have access to an input sequence. In this case, the input for the decoder at each time is just the decoder's output token from the previous time step.



Using a decoder during inference. The SOS token is always the initial decoder input token. For simplicity, the attention mechanism is not shown in this diagram.

In TensorFlow, we can switch from training to inference mode just by changing the `TrainingHelper` object to an inference helper. The most commonly used inference helper object is `GreedyEmbeddingHelper`.

B. Greedy decoding

The name of `GreedyEmbeddingHelper` comes from the fact that it utilizes a greedy decoding method and incorporates an embedding matrix.

The greedy decoding method means that the output at each decoder time step (also the input into the next time step) is just the highest probability vocabulary word at the time step. The output is converted into a word embedding prior to being used as input for the next time step.

Below we create a `GreedyEmbeddingHelper` with a vocabulary size of 598 and a batch size of 8.

```
import tensorflow as tf

vocab_size = 598
sos = vocab_size
eos = vocab_size + 1
extended_vocab_size = vocab_size + 2

# Placeholder representing the embedding matrix for the vocab
# Embedding dim is 10
embedding_matrix = tf.placeholder(
    tf.float32, shape=(600, 10)
)

batch_size = 8
start_tokens = tf.tile([sos], [batch_size])
helper = tf.contrib.seq2seq.GreedyEmbeddingHelper(
    embedding_matrix, start_tokens, eos)
```

The `GreedyEmbeddingHelper` is initialized with an embedding matrix, a `batch_size` length tensor of SOS tokens, and the EOS token. In the example above, we use the `tf.tile` function to create a length 8 tensor completely filled with the SOS token.

The (embedded) SOS token is always the initial decoder input token. It's used to give the decoder a starting point, and it marks the beginning of the decoded output. The EOS token marks the end of a decoded sequence. When the decoder returns the EOS token at some time step, the decoding process is terminated.

C. Embedding variable scope

When using feature columns to create embeddings (e.g. the `get_embeddings` function on line 35 in the code), we don't have direct access to the embedding matrix variable. This means we need to create an embedding matrix variable with the exact same name as the one used during training. To do this, we need to create the variable in the appropriate *variable scope*.

A variable scope is similar to directory paths in an operating system. We may know the name of the file (in this case the variable), but to retrieve it we have to be in the proper directory (the variable scope). We can also create a variable under a certain variable scope similar to how we would create files in certain directories.

We specify variable scopes in TensorFlow using the `tf.variable_scope` function. You'll notice that `tf.variable_scope` is used in the `get_embeddings` function.

```
import tensorflow as tf

with tf.variable_scope('scope1'):
    v1 = tf.get_variable('var', shape=(2, 2))

with tf.variable_scope('scope2'):
    v2 = tf.get_variable('var', shape=(2, 2))
```

The above code block creates variables named `'var'` inside scopes `'scope1'` and `scope2'`. Despite the shared name, these variables are different, since the scopes differ.

We can also use nested scopes to create a variable. A nested scope is similar to putting a file in subdirectories of outer directories.

```
import tensorflow as tf

with tf.variable_scope('s'):
    with tf.variable_scope('sub1'):
        v1 = tf.get_variable('v', shape=(2, 2))
    with tf.variable_scope('sub2'):
        v2 = tf.get_variable('v', shape=(2, 2))

print(v1)
```

```
print(v2)
```

Mimicking a nested scope using a single scope is also possible. We just need to use the appropriate "path" when specifying the scope name.

```
import tensorflow as tf

with tf.variable_scope('s/sub1'):
    v1 = tf.get_variable('v', shape=(2, 2))

print(v1)
```

The `tf.variable_scope` function also has a useful `reuse` keyword argument. While it's not used in the code for this course, it sets the argument with value `tf.AUTO_REUSE`, in order to re-use previously created variables.

We don't use it in the encoder-decoder model because the training and inference portions of the code are separate (i.e. we don't create the same embedding matrix twice in one model run).

D. Inference Decoding Translation Example

The code widget below uses inference decoding to translate english sentences into French. Type a sentence into the text box and it will print out the French translation.

```
translate()
```

Enter text here

Press Run To Execute

Time to Code!

In this chapter you'll be completing the `create_decoder_helper` function from the **Training Helper** chapter. This time, you'll be creating the Helper object for inference.

We've already filled in part of the code to obtain the decoder's embedding matrix and initialize the start and end tokens. Your job now is to create the `GreedyEmbeddingHelper` object for inference decoding.

Inside the `else` block of the `create_decoder_helper` function, set `helper` equal to `tf_s2s.GreedyEmbeddingHelper` applied with `embedding_weights`, `start_tokens`, and `end_token` as the three input arguments.

The `create_decoder_helper` function returns a tuple containing the helper object and ground truth sequence lengths. However, in inference mode there are no ground truth sequences. In this case, we just set `dec_seq_lens` equal to `None`.

Inside the `else` block, set `dec_seq_lens` equal to `None`.

```
import tensorflow as tf
tf_fc = tf.contrib.feature_column
tf_s2s = tf.contrib.seq2seq

# Seq2seq model
class Seq2SeqModel(object):
    def __init__(self, vocab_size, num_lstm_layers, num_lstm_units):
        self.vocab_size = vocab_size
        # Extended vocabulary includes start, stop token
        self.extended_vocab_size = vocab_size + 2
        self.num_lstm_layers = num_lstm_layers
        self.num_lstm_units = num_lstm_units
        self.tokenizer = tf.keras.preprocessing.text.Tokenizer(
            num_words=vocab_size)

    # Create the helper for decoding
    def create_decoder_helper(self, decoder_inputs, is_training, batch_size):
        if is_training:
            # ALREADY COMPLETED IN TRAINING HELPER CHAPTER
            pass
        else:
            DEC_EMB_SCOPE = 'decoder_emb/sequence_input_layer/sequences_embedding'
            with tf.variable_scope(DEC_EMB_SCOPE):
                embedding_weights = tf.get_variable(
                    'embedding_weights',
                    shape=(self.extended_vocab_size, int(self.extended_vocab_size**0.25)))
            start_tokens = tf.tile([self.vocab_size], [batch_size])
            end_token = self.vocab_size + 1
            # CODE HERE
```

```
        return helper, dec_seq_lens
```

Model Improvement

Learn strategies for improving an encoder-decoder model.

Chapter Goals:

- Learn strategies for improving an encoder-decoder model
- Run the encoder-decoder model in inference mode

A. Training strategies

Good encoder-decoder models tend to have a large number of weight parameters, since they consist of large LSTM/BiLSTM layers. Because of this, it usually takes a long time to train an encoder-decoder model to convergence.

To speed up training, it might be beneficial to use a larger batch size during the initial stages of training, then reduce the batch size once the model begins to show considerable improvement in reducing the loss function. A larger batch size allows the model to get through more data, and as long as we reduce the batch size at an early enough stage the model won't miss its convergence point.

Another strategy to speed up training is to start off with a larger learning rate, then gradually decrease the learning rate as the model trains.

B. Domain-specific strategies

The encoder-decoder model can be used for basically any seq2seq task. This includes domains like machine translation, text summarization, and dialog systems (e.g. chatbots). However, each of these domains has their own special features, which may require tweaks to how the encoder-decoder model is used.

For example, in text summarization it is often a good idea to truncate the input text to a maximum length. This is because input text can be incredibly long (think huge news articles) and also because much of the important information in a text can be found in the first few paragraphs. Similarly, we

would want to set a maximum decoding length when creating summaries, so that the summaries don't just replicate the entire article.

When training a dialog system such as a chatbot, the data we use and process is incredibly important. For example, we would want to train a customer service chatbot on data that contains dialogue specific to customer service. Furthermore, we need to be careful to prune out profanity and other words that we don't want the dialog system to output.

In general, it is important to understand the seq2seq task's domain prior to building an encoder-decoder model for the task. This way you can make the necessary data and model tweaks that will generate the best performance.

Datasets

An overview of some useful datasets to train a seq2seq model with.

A. Seq2seq datasets

With the encoder-decoder model architecture, we have a model template for any sequence to sequence NLP task. Of course, each seq2seq task will require some tweaks to the model, but overall the architecture largely stays the same across all domains.

The most important part to creating a good seq2seq model is finding a lot of useful training data. For machine translation, there are tons of training datasets online. You can find many of the largest machine translation datasets here.

For training a dialog system such as a chatbot, a couple good datasets to use are the Ubuntu Dialogue Corpus and the Cornell Movie Corpus.

For text summarization, a good training set to use is the CNN/Daily Mail summarization dataset. Instructions for how to obtain the data can be found here.

Quiz

1

What’s the main purpose of using an encoder-decoder model?

COMPLETED 0%

1 of 4

Course Conclusion

A. Closing thoughts

Natural language processing is one of the hottest fields in industry, since nearly every company needs to provide meaningful results and insights from raw text data. From social media and finance to housing and retail, many companies across a variety of different industries are leveraging ML-based natural language processing techniques to get ahead.

B. Course recap

This course has provided an overview of how to use machine learning to perform various natural language processing tasks. You'll be able to take the practical lessons and actionable insights from this course and apply them to your projects.

C. Topics covered

- Text processing
- Word embeddings
- sentiment analysis
- machine translation
- text generation.

D. Industry standard frameworks used in this course

- NumPy
- TensorFlow
- Keras